Chapter 6: Sequence Prediction

Prachya Boonkwan (Arm)

School of ICT, SIIT,
Thammasat University

prachya@siit.tu.ac.th, kaamanita@gmail.com

Who? Me?

- Nickname: **Arm** (P'/N' Arm, etc.)
- Born: Aug 1981
- Work
  - Researcher at NECTEC 2005-2024
  - Lecturer at SIIT, Thammasat University 2025-onw
- Education
  - B.Eng & M.Eng, CPE Kasetsart University
  - Obtained Ministry of Science Scholarship in early 2008
  - Did a PhD in Informatics (AI & Computational Linguistics) at University of Edinburgh, UK from 2008 to 2013 (4.5 years)

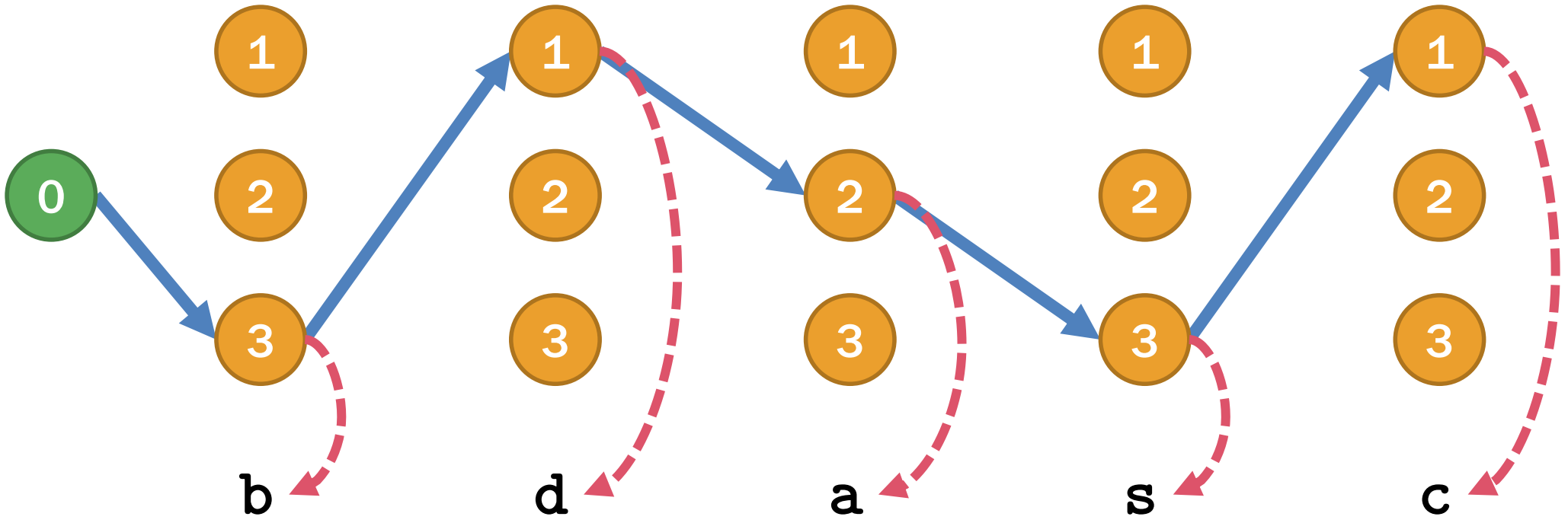
Outline

- Sequence prediction
- Expectation Maximization Algorithm
- Hidden Markov models
- Tree prediction
- Noise reduction
- Conclusion

Sequence Prediction

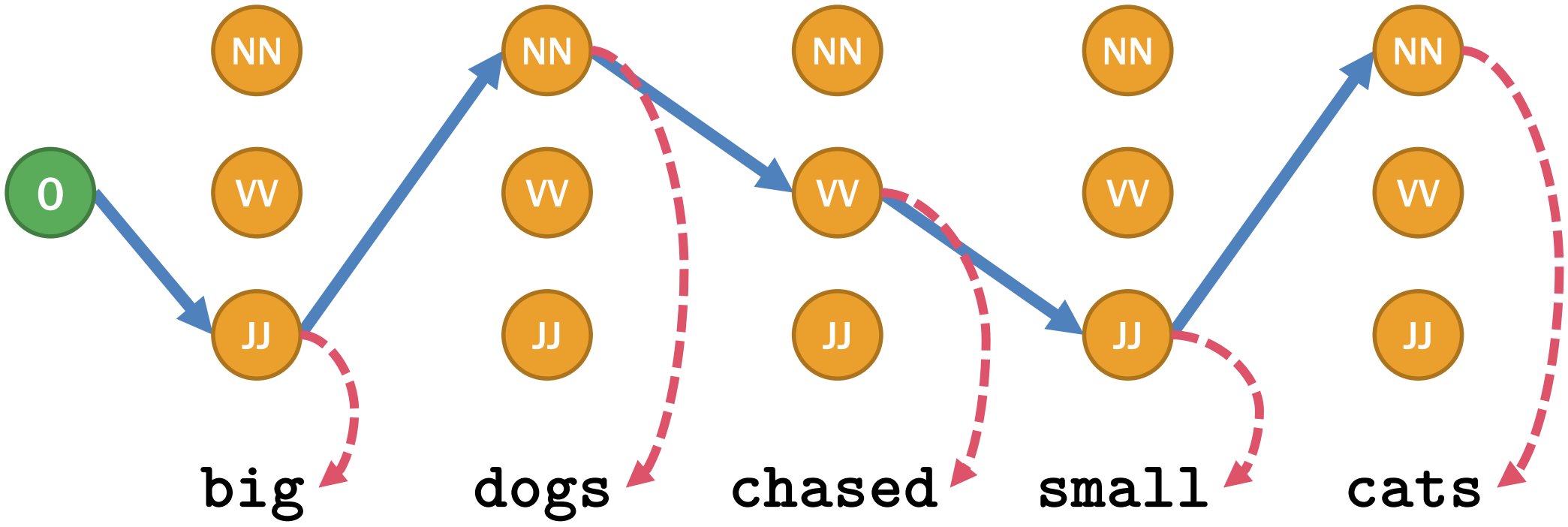
Sequence Prediction

- We assume that a given symbolic string is statistically generated by a hidden sequence of state transitions



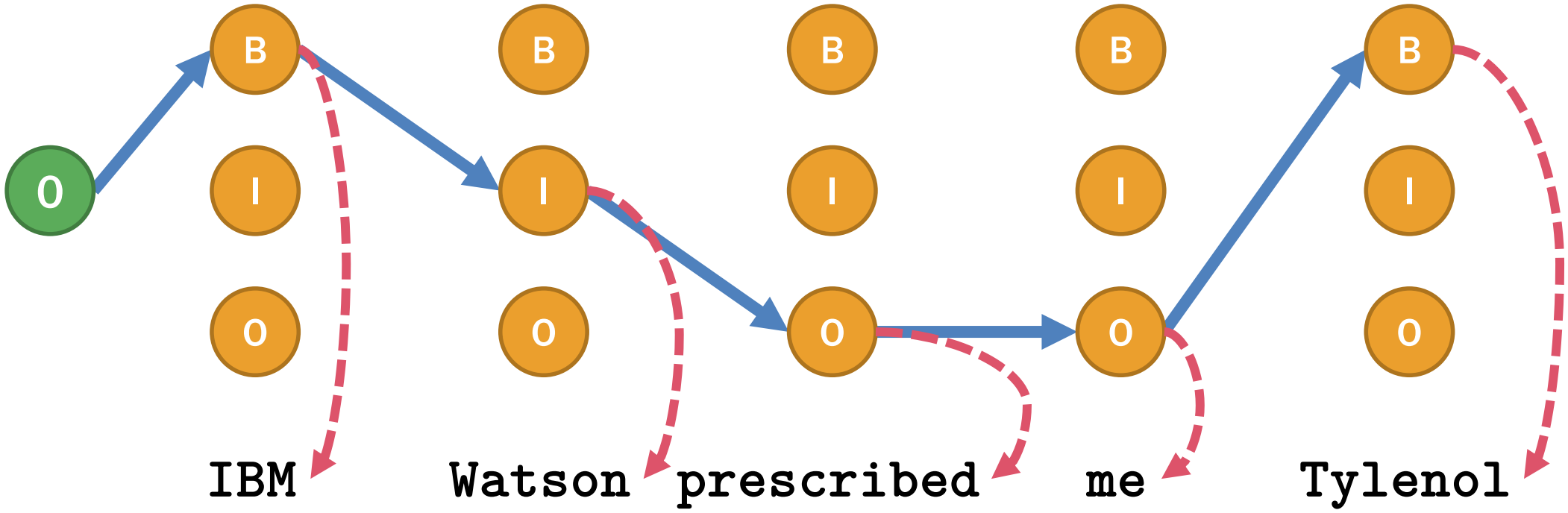
Sequence Prediction

- Several NLP tasks can be modeled as sequence prediction
  - E.g. POS tagging



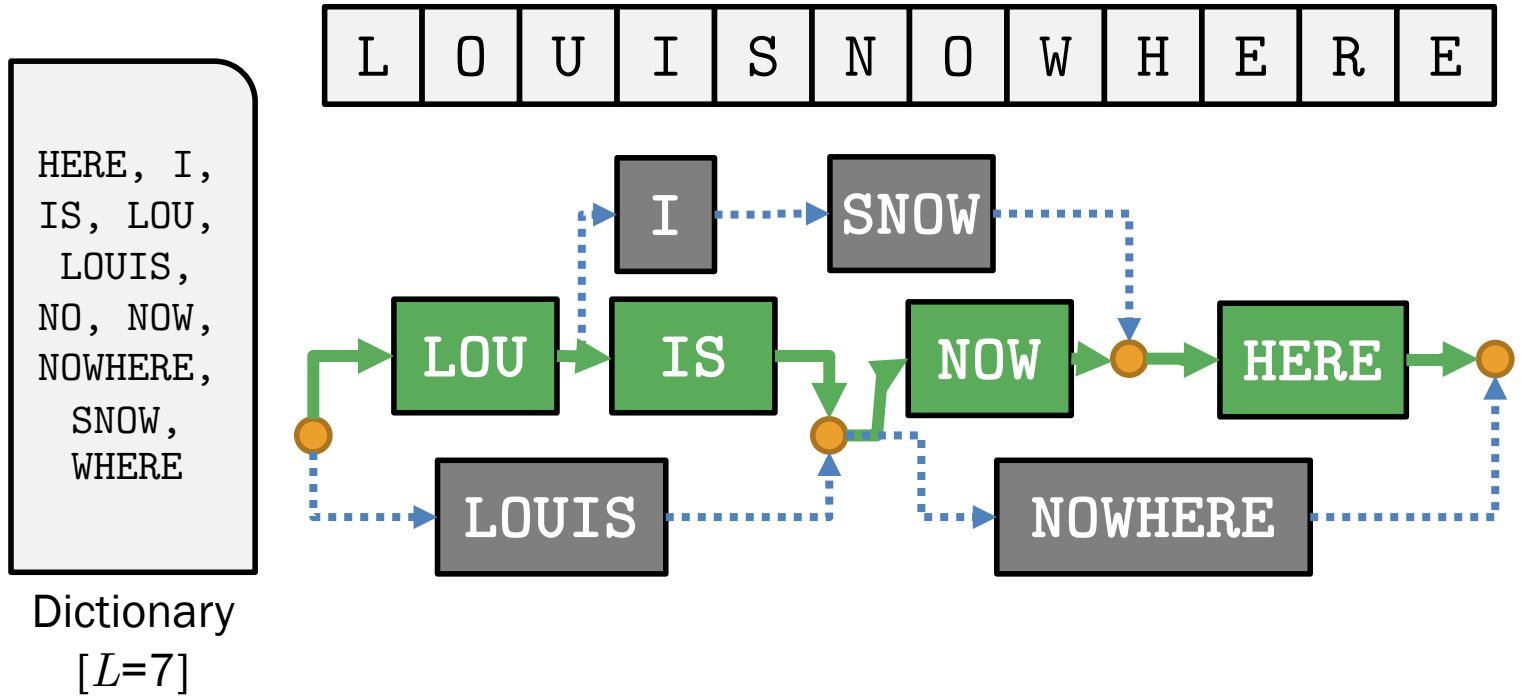
Sequence Prediction

- Several NLP tasks can be modeled as sequence prediction
  - E.g. named entity recognition [B = begin, I = intermediate, O = outer]



Sequence Prediction

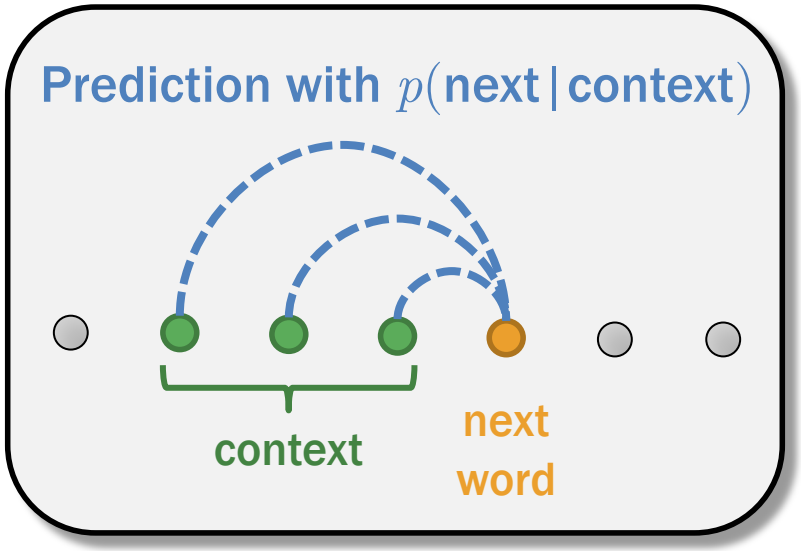
- Several NLP tasks can be modeled as sequence prediction
  - E.g. word segmentation [each word becomes a state itself]



Language Model

- Statistical prediction for how strings are produced in a language
- Interpreted as a generative model
  1. Generate the first word w_1
  2. Keep generating the **next word** w_k based on the previous words (a.k.a. **context**) w_1 ... w_{k-1} until the whole sentence of length N is produced

$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^{N} p(\underbrace{w_k}_{\text{next word}} | \underbrace{w_1 \dots w_{k-1}}_{\text{context}})$$



We need at least 1 billion words of text data to train a stable LLM, which learns **word collocations** and **phrase structures**

Hidden Markov Model (HMM)

- Statistical prediction of the next event based on only the recent events
  - Alleviating the sparse distribution of $p(\text{next word} \mid \text{context})$ by truncation to $p(\text{next word} \mid \text{previous})$

next word context

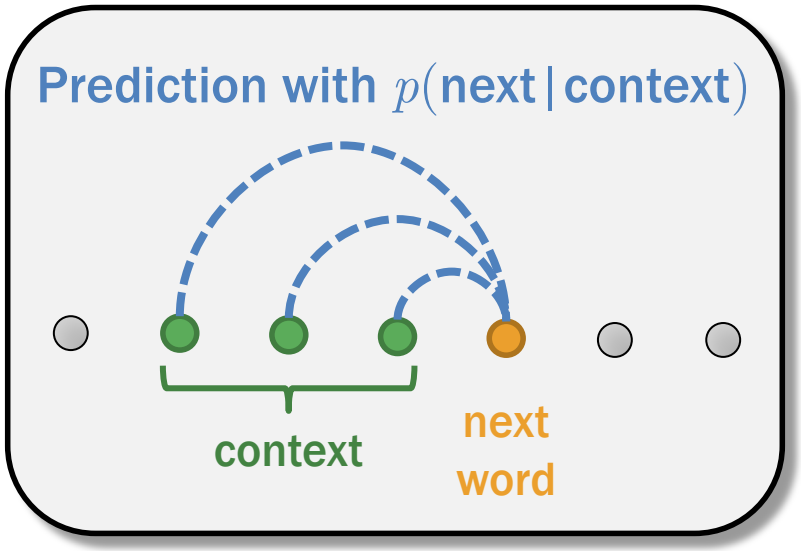
$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^{N} p(w_k | w_1 \dots w_{k-1})$$

$$\approx p(w_1) \prod_{k=2}^{N} p(w_k | w_{k-1})$$

Truncation is sometimes called the **Markov's blanket**

n-Gram Model

- Language models whose context is truncated to at most $n-1$ previous words



$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^{N} p(\overbrace{w_k}^{\text{next word}} | \overbrace{w_{k-n+1} \dots w_{k-1}}^{n-1 \text{ prev words}})$$

- **Unigram** $(n=1)$: $P(w_1 \dots w_N) = \prod_{k=1}^{N} p(w_k)$
- **Bigram** $(n=2)$: $P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^{N} p(w_k|w_{k-1})$
- **Trigram** $(n=3)$: $P(w_1 \dots w_N) = p(w_1)p(w_2|w_1) \prod_{k=3}^{N} p(w_k|w_{k-2}, w_{k-1})$

Expectation Maximization Algorithm

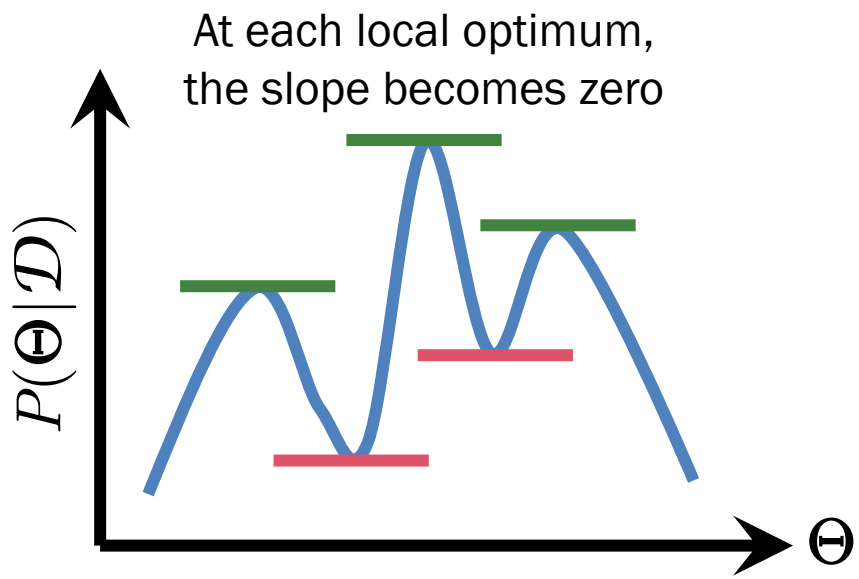
Model Optimization

- Finding a set of parameter values Θ^* that maximize the posterior $P(\Theta|\mathcal{D})$, i.e. an optimal hypothesis

$$\Theta^* = \arg\max_{\Theta} P(\Theta|\mathcal{D})$$

$$\frac{\partial}{\partial \Theta} P(\Theta|\mathcal{D}) = 0$$

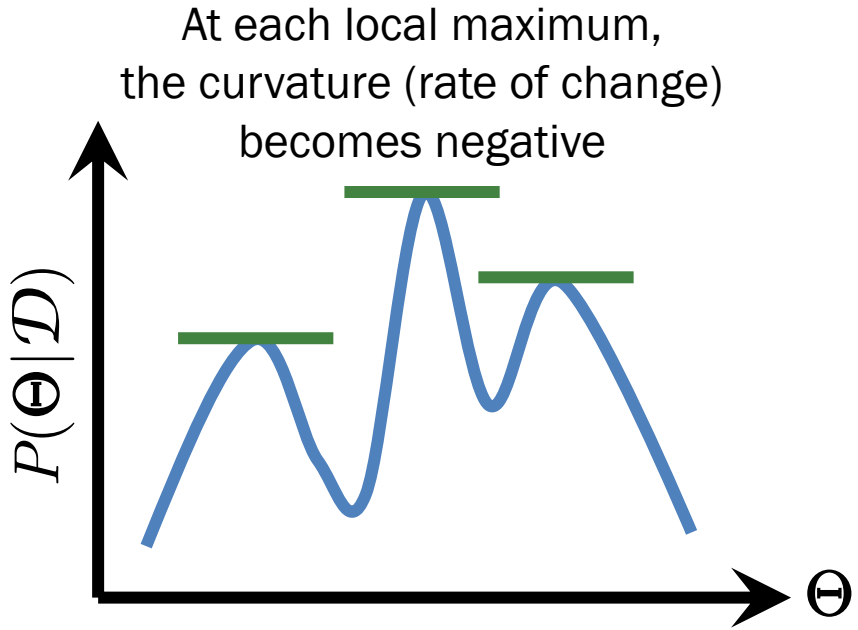
(slope)



Model Optimization

- Finding a set of parameter values Θ^* that maximize the posterior $P(\Theta|\mathcal{D})$, i.e. an optimal hypothesis

$$\Theta^* = \arg\max_{\Theta} P(\Theta|\mathcal{D})$$



$$\frac{\partial}{\partial\Theta} P(\Theta|\mathcal{D}) = 0$$

(slope)

and

$$\frac{\partial^2}{\partial\Theta^2} P(\Theta|\mathcal{D}) < 0$$

(curvature)

Finding Θ^* from $P(\Theta|\mathcal{D})$ is hard

- Suppose the dataset $\mathcal{D} = \{x_1, x_2, x_3, \ldots, x_n\}$
- We **cannot** factorize $P(\Theta|x_1, x_2, x_3, \ldots, x_n)$ into $P(\Theta|x_1)$, $P(\Theta|x_2)$, $P(\Theta|x_3)$, ..., and $P(\Theta|x_n)$
- However, we can translate $P(\Theta|x_1, x_2, x_3, \ldots, x_n)$ into $P(x_1, x_2, x_3, \ldots, x_n|\Theta)$ via Bayes' rule and easily factorize this term

$$
\begin{aligned}
&P(\Theta|x_1, x_2, x_3, \ldots, x_n) \\
&\quad \propto P(x_1, x_2, x_3, \ldots, x_n|\Theta)\ P(\Theta) \\
&\quad = P(x_1|\Theta)\ P(x_2|\Theta)\ P(x_3|\Theta)\ \ldots\ P(x_n|\Theta)\ P(\Theta)
\end{aligned}
$$

Maximum Likelihood Estimation (MLE)

- We find optimal parameters by assuming that the prior probability $P(\Theta)$ is uniform [i.e. no prior knowledge]

$$\begin{aligned} P(\Theta|x_1,x_2,x_3,\ldots,x_n) & \qquad \text{uniform distribution} \\ \propto\ & P(x_1,x_2,x_3,\ldots,x_n|\Theta)\ \cancel{P(\Theta)} \\ =\ & P(x_1,x_2,x_3,\ldots,x_n|\Theta) \end{aligned}$$

- Let $\mathcal{L}$ be the likelihood function of the parameters Θ given the dataset $\mathcal{D}$ [we want to maximize it]

$$\mathcal{L}(\Theta;\mathcal{D}) = P(x_1, x_2, x_3, \ldots, x_n|\Theta)$$

Maximum A Posteriori (MAP)

- We find optimal parameters by assuming that the prior probability $P(\Theta)$ is not uniform [i.e. we have prior knowledge]

$$P(\Theta|x_1,x_2,x_3,\ldots,x_n) \propto P(x_1,x_2,x_3,\ldots,x_n|\Theta)\ P(\Theta)$$

- Let $\mathcal{L}$ be the likelihood function of the parameters Θ given the dataset $\mathcal{D}$ [we want to maximize it]

$$\mathcal{L}(\Theta;\mathcal{D}) = P(x_1, x_2, x_3, \ldots, x_n|\Theta)P(\Theta)$$

Unobserved Variables

- In practice, a model consists of observed parameters Θ (things we can observe) and unobserved variables $\mathbf{Z}$ (things beyond our observation, a.k.a. noise)

$$\mathcal{L}(\boldsymbol{\Theta};\mathcal{D}) = \sum_{\mathbf{Z}} \tilde{\mathcal{L}}(\boldsymbol{\Theta},\mathbf{Z};\mathcal{D}) = \int \tilde{\mathcal{L}}(\boldsymbol{\Theta},\mathbf{Z};\mathcal{D})d\mathbf{Z}$$

- Eliminating $\mathbf{Z}$ by marginalization is intractable because it could be an intricate series of dependent events

Expectation-Maximization Algorithm (Dempster et al., 1977)

- Iterative method to find MLE and MAP
  - **Assumption:** we can eliminate noise $\mathbf{Z}$ by computing an expectation over a large enough dataset
  - **E-Step:** Compute the expectation $\mathbf{Q}$ of log-likelihood given the current estimate of parameters $\boldsymbol{\Theta}$

$$\mathbf{Q}(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = \mathbb{E}_{P(\mathbf{Z}|\mathcal{D},\boldsymbol{\Theta}^{(t)})}\left[\log \mathcal{L}(\boldsymbol{\Theta};\mathcal{D},\mathbf{Z})\right]$$

  - **M-Step:** Find the parameters that maximize this expectation

$$\boldsymbol{\Theta}^{(t+1)} = \arg\max_{\boldsymbol{\Theta}} \mathbf{Q}(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)})$$

Don't worry about the calculus here. We will learn only their analytical results in this class.

EM Algorithm in Practice

- We iterate the E and M steps until the parameters do not change anymore
  - **E-Step:** we design a dynamic programming algorithm that efficiently computes the expectation for each parameter in Θ

$$\mathbf{Q}(\Theta|\Theta^{(t)}) = \boxed{\text{Dynamic Programming}}$$

Sometimes we call it the $\mathbf{Q}$ function

  - **M-Step:** we recompute a new value for each parameter from the computed expectations

$$\Theta^{(t+1)} = \boxed{\text{Estimation from } \mathbf{Q}}$$

Usually, arithmetic division will do, but other techniques (e.g. MAP) can eliminate the noise

Simplifying EM: the case of MLE

- Each instance in dataset $\mathcal{D}$ is (a,b,c)
- Each parameter θ_{abc} in Θ is for $P(c|a,b)$
- MLE: No prior for each θ_{abc}

- The dataset likelihood is

$$\mathcal{L}(\Theta|\mathcal{D}) = \prod_{a,b,c} \theta_{abc}^{\#(a,b,c)}$$

```
repeat:
    let p and q be empty hash tables
    for each instance (a,b,c) in D:
        set p[a,b,c] += θ[a,b,c]      # expectation of (a,b,c)
        set q[a,b] += θ[a,b,c]        # expectation of (a,b)
    for each key (a,b,c) in θ:
        set θ[a,b,c] = p[a,b,c] / q[a,b]   # maximizing θ
until Θ converges
```

Simplifying EM: the case of MAP

- Each instance in dataset $\mathcal{D}$ is (a,b,c)
- Each parameter θ_{abc} in Θ is for $P(c|a,b)$
- MAP: Dirichlet prior for each θ_{abc} is α_{abc}

- The dataset likelihood is

$$\mathcal{L}(\Theta|\mathcal{D}) = \prod_{a,b,c} \theta_{abc}^{\#(a,b,c)+\alpha_{abc}-1}$$

```
repeat:
    let p and q be empty hash tables
    for each instance (a,b,c) in D:
        set p[a,b,c] += θ[a,b,c]
        set q[a,b] += θ[a,b,c]
    for each key (a,b,c) in p:
        set θ[a,b,c] = (p[a,b,c] + α[a,b,c]) / (q[a,b] + ∑_c α[a,b,c])
until Θ converges
```

In this class

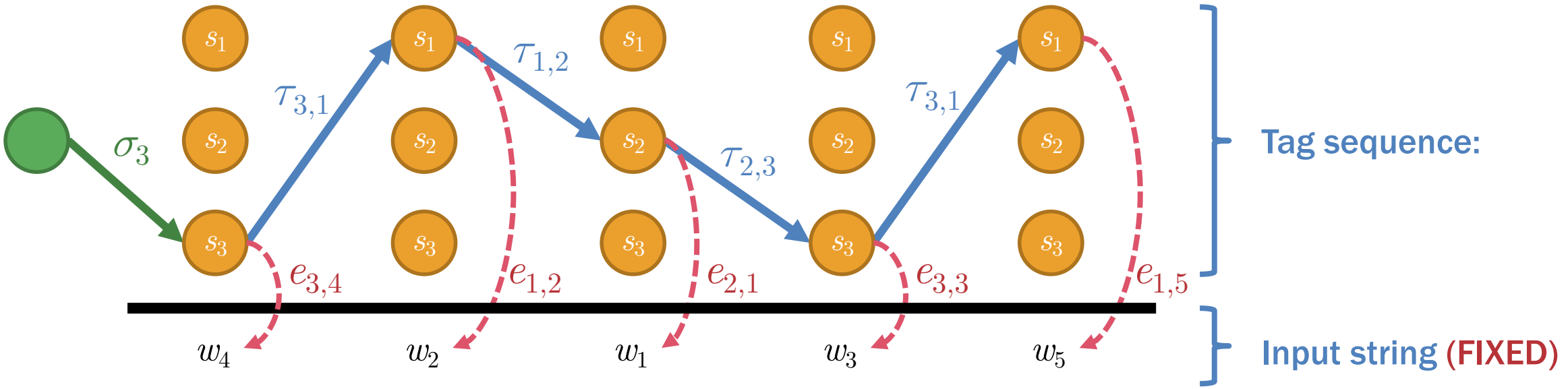
- We will learn two basic algorithms that follow the EM Algorithm
  - Sequence prediction with hidden Markov models
    - Computing path expectations from overlapping state transitions using dynamic programming
    - Estimating the parameters by arithmetic division
  - Tree prediction with probabilistic CFGs
    - Computing tree expectations from overlapping subtrees using dynamic programming
    - Estimating the parameters by arithmetic division

1. Parameter Estimation with EM

(Baum and Petrie, 1966; Baum and Welch, 1960s)

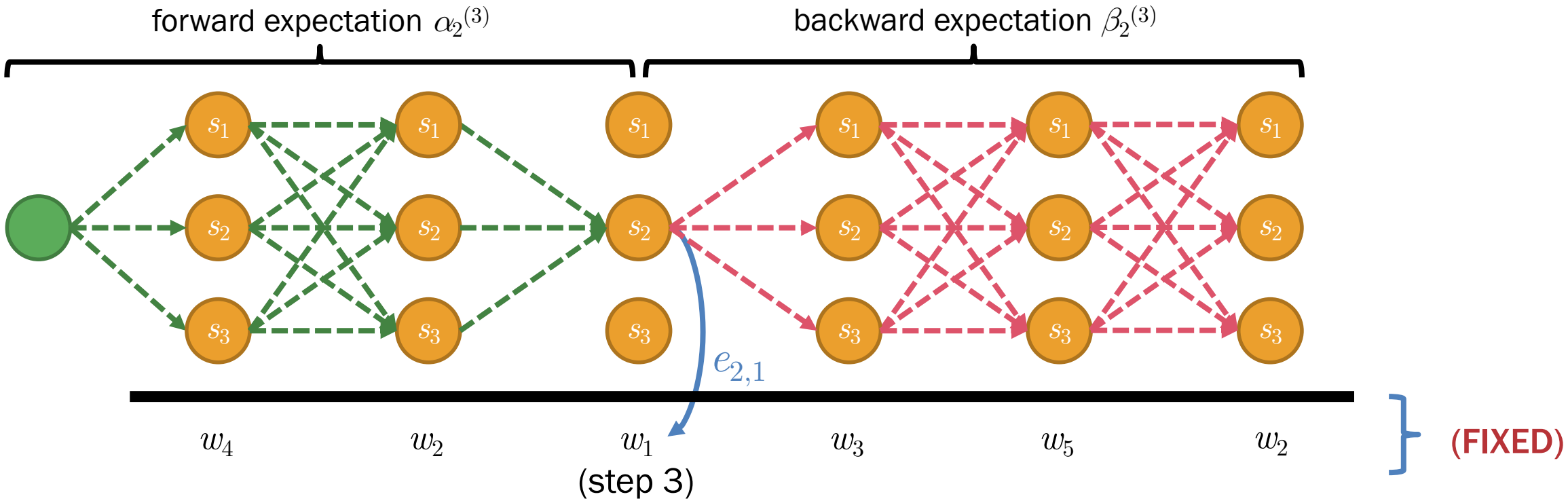
Hidden Markov Model (HMM)

- Assuming three conditional probabilities as parameters
  - **Start probability:** $\sigma_x = P(s_x \mid \text{START})$
  - **Transition probability:** $\tau_{xy} = P(s_y \mid s_x)$
  - **Emission probability:** $e_{xi} = P(o = w_i \mid s_x)$



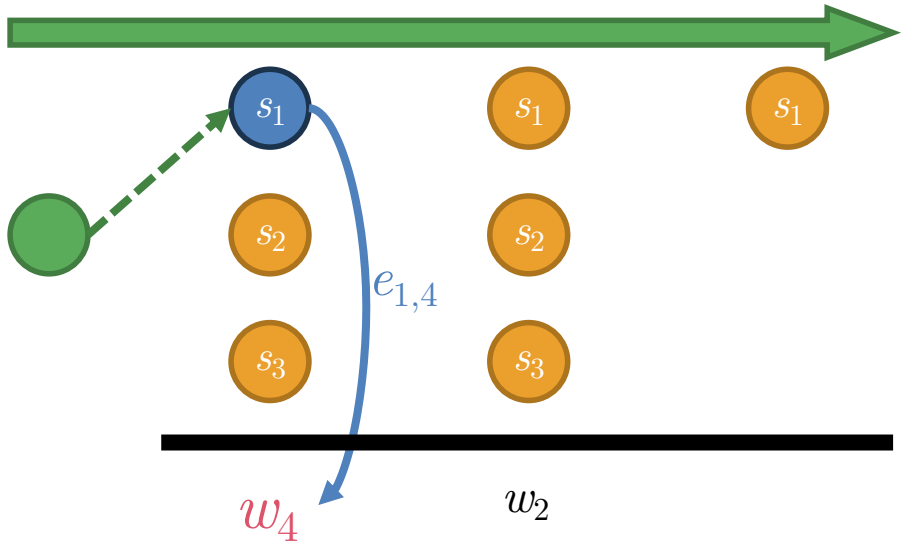
Q Function: State Expectations

- At step t, there are 2 state expectations
  - **Forward expectation** $\alpha_x^{(t)}$ at state s_x
  - **Backward expectation** $\beta_y^{(t)}$ at state s_y



Computing Forward Expectation

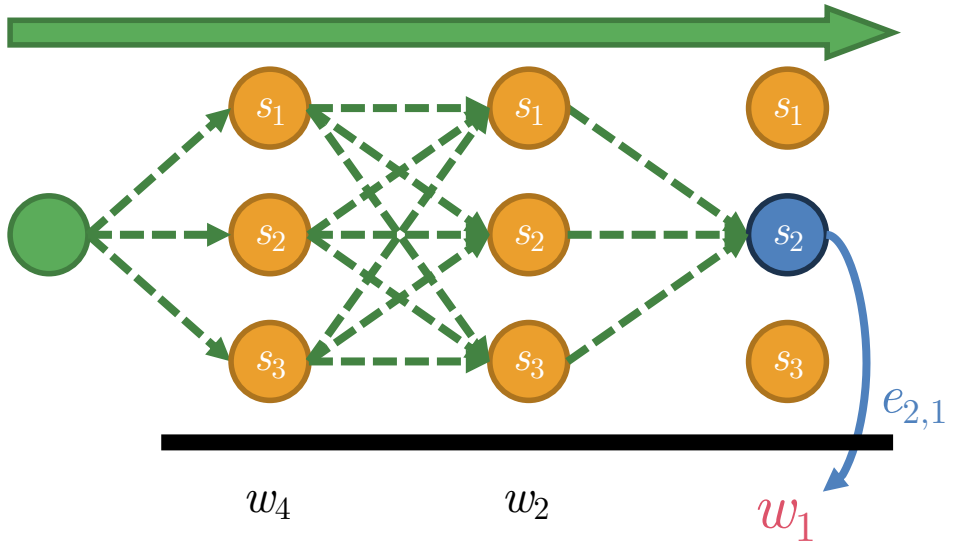
- Base case
  - $\alpha_{\text{curr state } x}{}^{(1)} = e_{x,\text{ word } i}\,\sigma_x$
- Recursive case
  - $\alpha_{\text{curr state } x}{}^{(t)} = e_{x,\text{ word } i} \sum_{\text{prev state } y} [\tau_{yx}\,\alpha_y{}^{(t\text{-}1)}]$, where $o_t = w_i$



$$\alpha_1{}^{(t=1)} = e_{1,4}\,\sigma_1$$

Computing Forward Expectation

- Base case
  - $\alpha_{\text{curr state } x}{}^{(1)} = e_{x,\text{ word } i}\, \sigma_x$



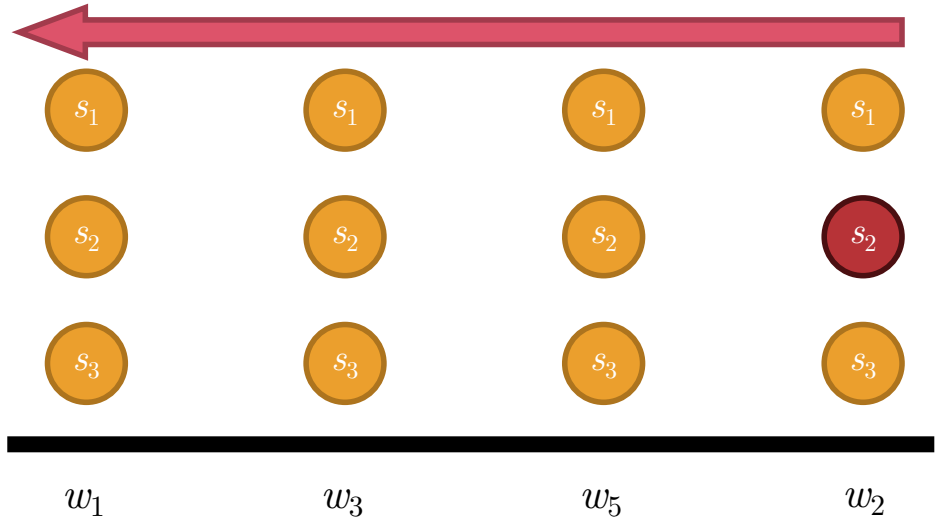
- Recursive case
  - $\alpha_{\text{curr state } x}{}^{(t)} = e_{x,\text{ word } i} \sum_{\text{prev state } y} [\tau_{yx}\, \alpha_y{}^{(t-1)}]$, where $o_t = w_i$

$$\alpha_2{}^{(t=3)} = e_{2,1} [\ \tau_{1,2}\, \alpha_1{}^{(t=2)} + \tau_{2,2}\, \alpha_2{}^{(t=2)} + \tau_{3,2}\, \alpha_3{}^{(t=2)}\ ]$$

Computing Backward Expectation

- **Base case**
  - $\beta_{\text{curr state } y}^{(N)} = 1$

- Recursive case
  - $\beta_{\text{curr state } y}^{(t)} = \sum_{\text{next state } z} [e_{z,\text{ next word } j}\, \tau_{yz}\, \beta_z^{(t+1)}]$, where $o_{t+1} = w_j$

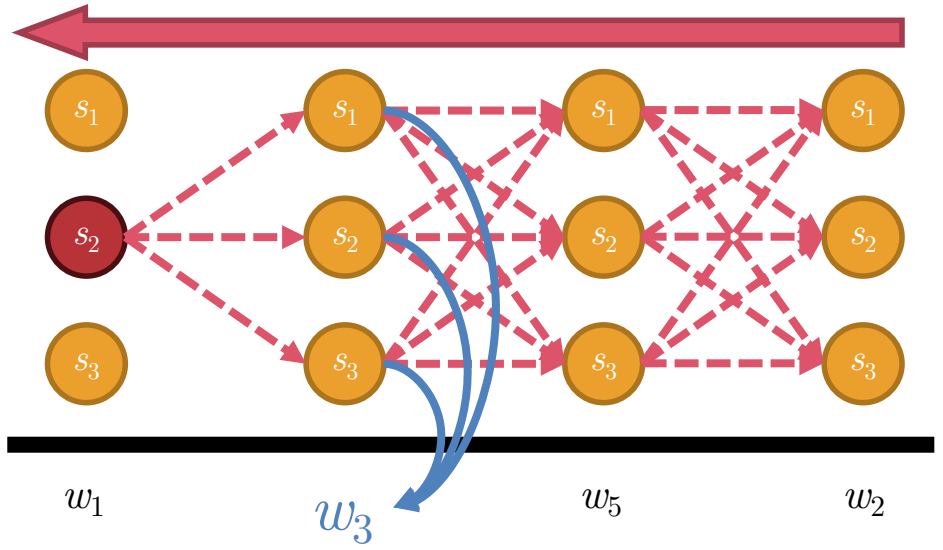


$$\beta_2^{(t=6)} = 1$$

Computing Backward Expectation

- Base case
  - $\beta_{\text{curr state } y}^{(N)} = 1$

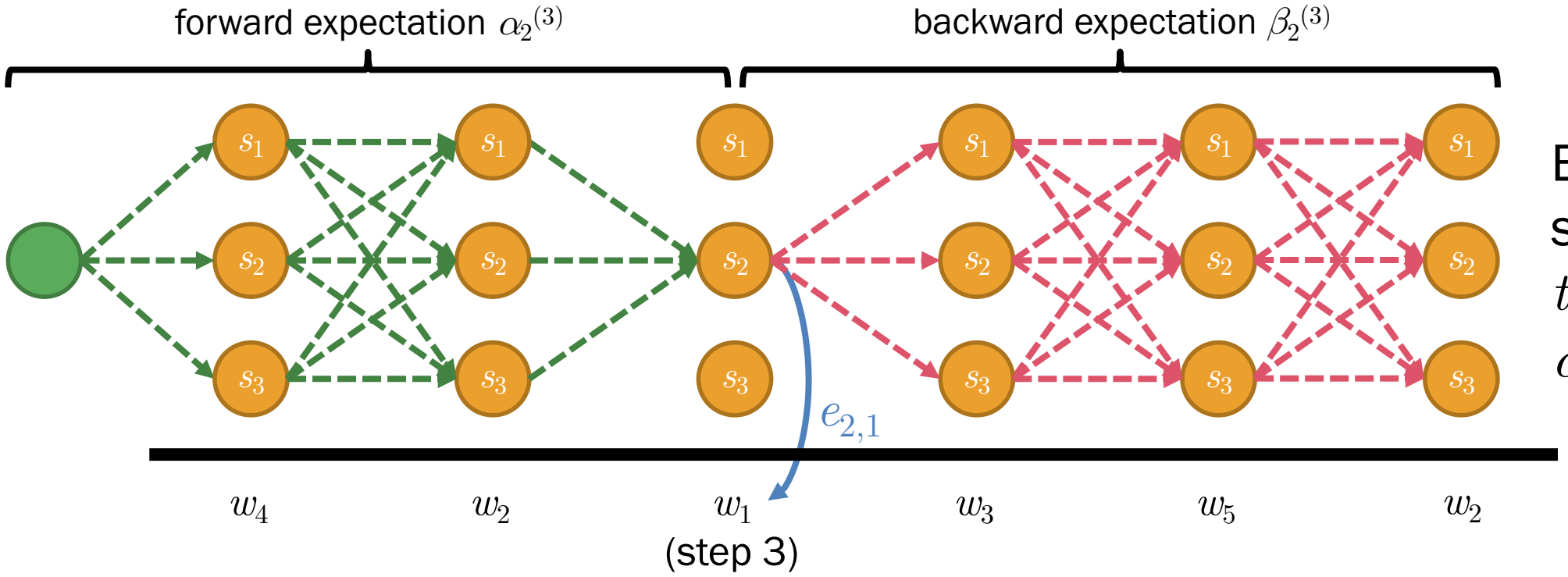
- **Recursive case**
  - $\beta_{\text{curr state } y}^{(t)} = \sum_{\text{next state } z} [e_{z,\text{ next word } j}\ \tau_{yz}\ \beta_z^{(t+1)}]$, where $o_{t+1} = w_j$



$$\beta_2(t{=}3) = e_{1,3}\ \tau_{2,1}\ \beta_1(t{=}4) + e_{2,3}\ \tau_{2,2}\ \beta_2(t{=}4) + e_{3,3}\ \tau_{2,3}\ \beta_3(t{=}4)$$

Q Function: State Expectations

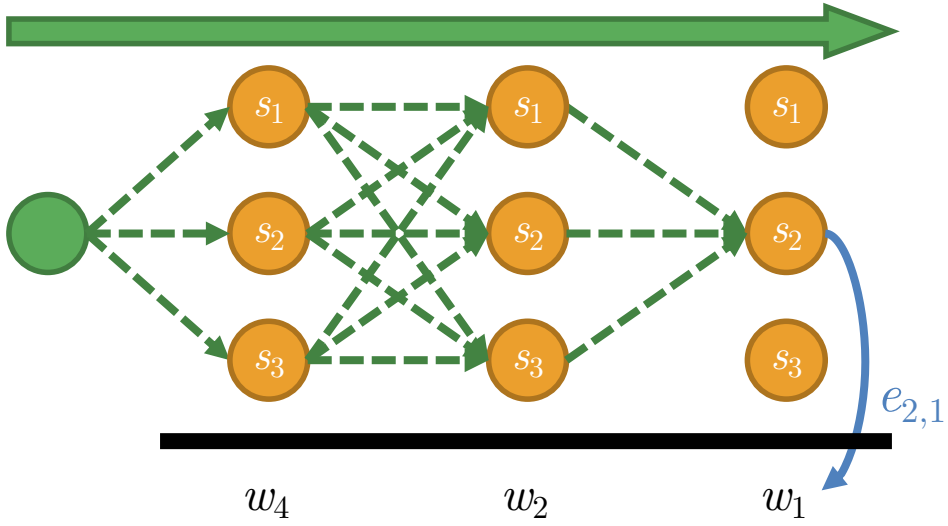
- These expectations can be defined recursively
  - **Forward expectation** $\alpha_x^{(t)} = e_{xi} \sum_y [\tau_{yx}\, \alpha_y^{(t-1)}]$, where $o_t = w_i$
  - **Backward expectation** $\beta_x^{(t)} = \sum_y [e_{yj}\, \tau_{xy}\, \beta_y^{(t+1)}]$, where $o_{t+1} = w_j$



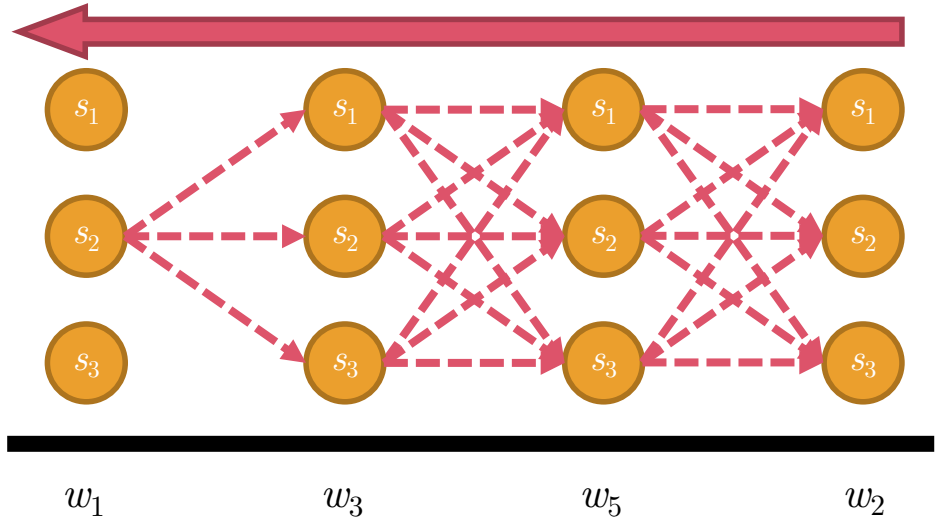
Expectation of state s_x at step t is therefore $\alpha_x^{(t)}\, \beta_x^{(t)}$

Q Function: State Expectations

- **Forward expectation** α
  - Base case: $\alpha_x^{(1)} = e_{xi}\,\sigma_x$ (start state)
  - Recursive case: $\alpha_x^{(t)} = e_{xi} \sum_y [\tau_{yx}\,\alpha_y^{(t-1)}]$, where $o_t = w_i$

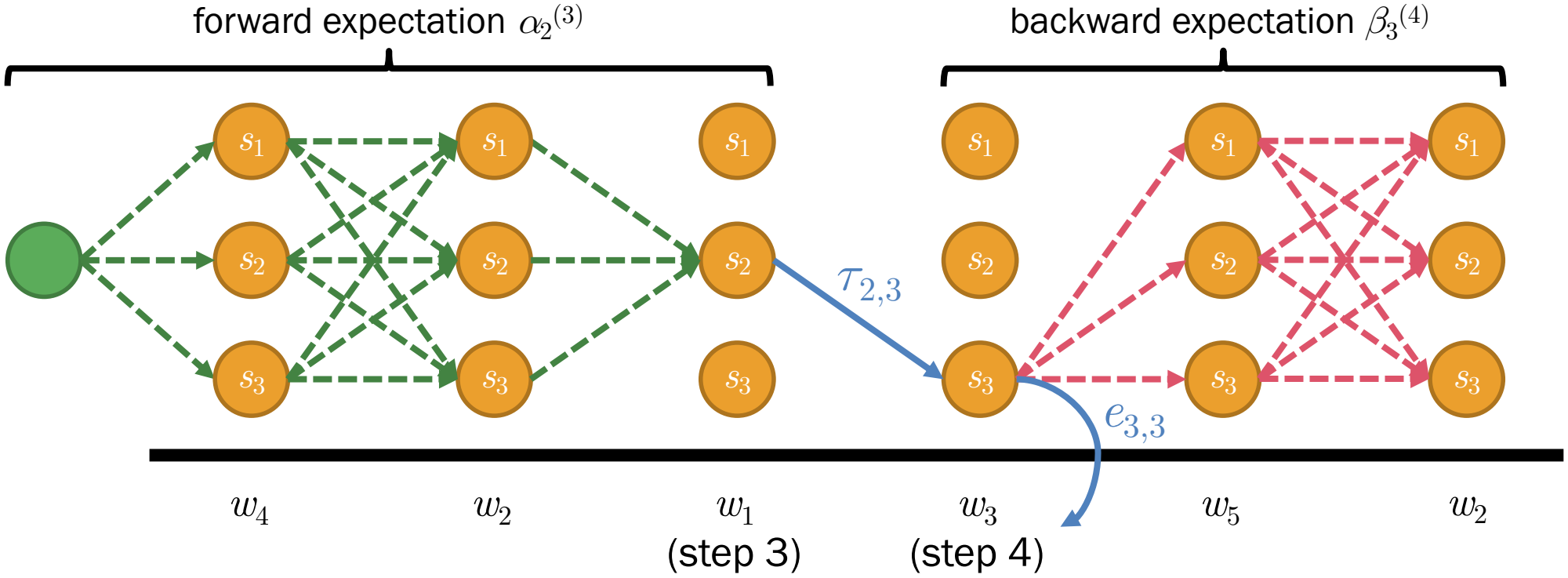


- **Backward expectation** β
  - Base case: $\beta_y^{(N)} = 1$ (final state)
  - Recursive case: $\beta_y^{(t)} = \sum_z [e_{zj}\,\tau_{yz}\,\beta_z^{(t+1)}]$, where $o_{t+1} = w_j$



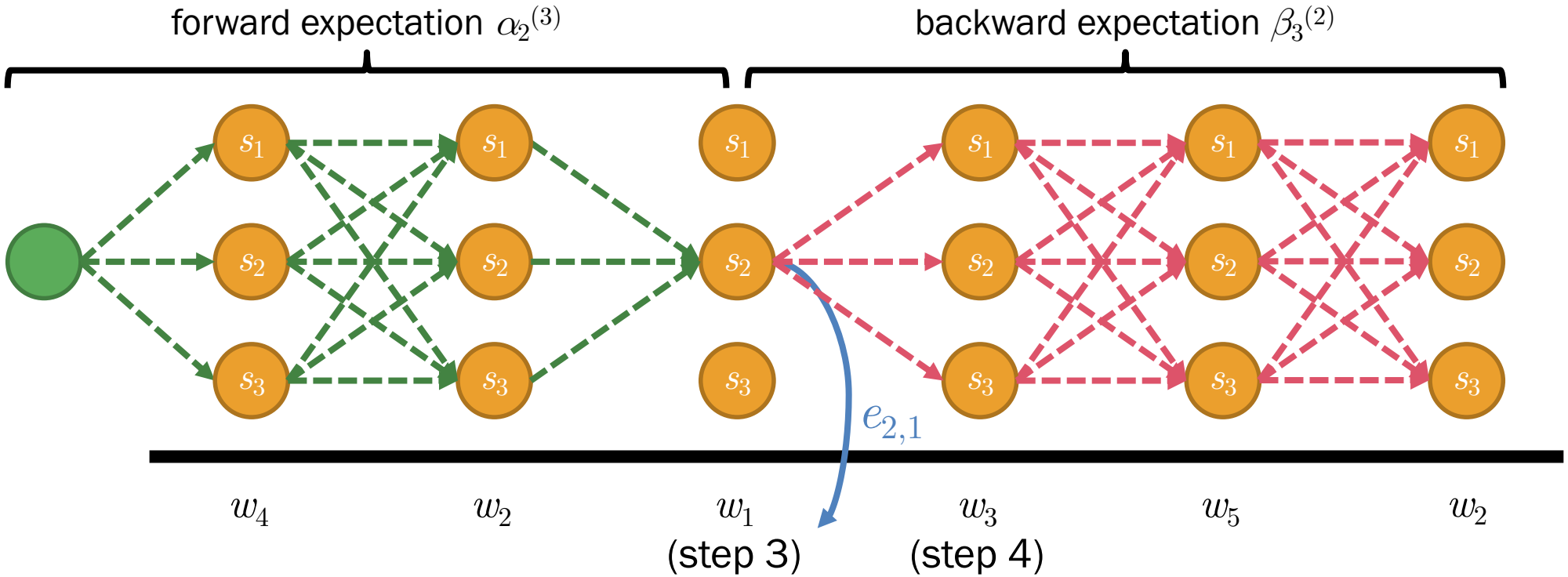
E-Step: Expectation of State Transition

- Multiplication of $\alpha_x^{(t)}$ (going from START to state s_x), τ_{xy} (transitioning from s_x to s_y), e_{yi} (emitting observation w_i at state s_y), and $\beta_y^{(t+1)}$ (going from state s_y to all final states)



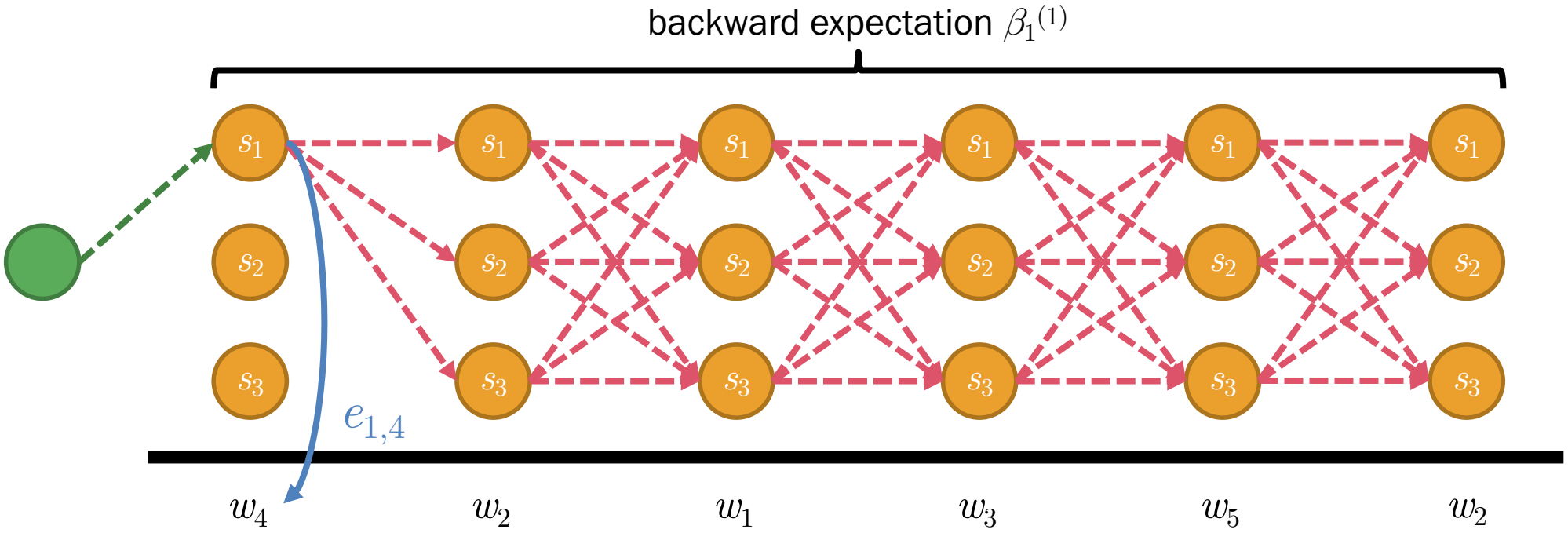
E-Step: Expectation of Emission

- Multiplication of $\alpha_x^{(t)}$ (going from START to state s_x), e_{xi} (emitting observation w_i at state s_x), and $\beta_x^{(t)}$ (going from state s_x to all final states)



E-Step: Expectation of Starting

- Multiplication of e_{xi} (emitting observation w_i at state s_x), and $\beta_x^{(1)}$ (going from state s_x to all final states)



M-Step: Update Equations

- Update the parameters with the following equations
  - State transition $\hat{\tau}_{xy} = p(s_y|s_x)$

$$= \frac{\sum_t \sum_i \alpha_x^{(t)} \tau_{xy} e_{yi} \beta_y^{(t+1)}}{\sum_x \sum_t \sum_i \alpha_x^{(t)} \tau_{xy} e_{yi} \beta_y^{(t+1)}}$$

  - Emission $\hat{e}_{xi} = p(w_i|s_x)$

$$= \frac{\sum_t \sum_y \alpha_x^{(t)} \tau_{xy} e_{yi} \beta_y^{(t+1)}}{\sum_x \sum_t \sum_y \alpha_x^{(t)} \tau_{xy} e_{yi} \beta_y^{(t+1)}}$$

  - Starting $\hat{\sigma}_x = p(s_x|\text{START})$

$$= \frac{\sum_i e_{xi} \beta_x^{(1)}}{\sum_x \sum_i e_{xi} \beta_x^{(1)}}$$

Forward-Backward Algorithm (MLE)

repeat:
    let p_σ, p_τ, p_e, q_σ, q_τ, q_e be empty hash tables and $q_\sigma = 0$
    for each string $\langle (w_1,s_1), \ldots, (w_N,s_N) \rangle$ in $\mathcal{D}$:
        compute forward expectation α
        compute backward expectation β
        for t in $[1,N-1]$:
            update $p_\tau[s_t,s_{t+1}]$ += $\alpha[s_t]^{(t)}\ \tau[s_t,s_{t+1}]\ e[s_{t+1},w_{t+1}]\ \beta[s_{t+1}]^{(t+1)}$ # exp. of transition
            update $q_\tau[s_t]$ += $\alpha[s_t]^{(t)}\ \tau[s_t,s_{t+1}]\ \beta[s_{t+1}]^{(t+1)}$
        for t in $[1,N]$:
            update $p_e[s_t,w_t]$ += $\alpha[s_t]^{(t)}\ e[s_t,w_t]\ \beta[s_t]^{(t)}$ # exp. of emission
            update $q_e[s_t]$ += $\alpha[s_t]^{(t)}\ e[s_t,w_t]\ \beta[s_t]^{(t)}$
        update $p_\sigma[s_t]$ += $\beta[s_t]^{(1)}$ # exp. of starting
        update q_σ += $\beta[s_t]^{(1)}$
    for each key (s_x,s_y) in τ: set $\tau[s_x,s_y] = p_\tau[s_x,s_y] \ / \ q_\tau[s_x]$ # maximizing τ
    for each key (s_x,w_i) in e: set $e[s_x,w_i] = p_e[s_x,w_i] \ / \ q_e[s_x]$ # maximizing e
    for each key s_x in σ: set $\sigma[s_x] = p_\sigma[s_x] \ / \ q_\sigma$ # maximizing σ
until σ, τ, and e converge

Forward-Backward Algorithm (MAP)

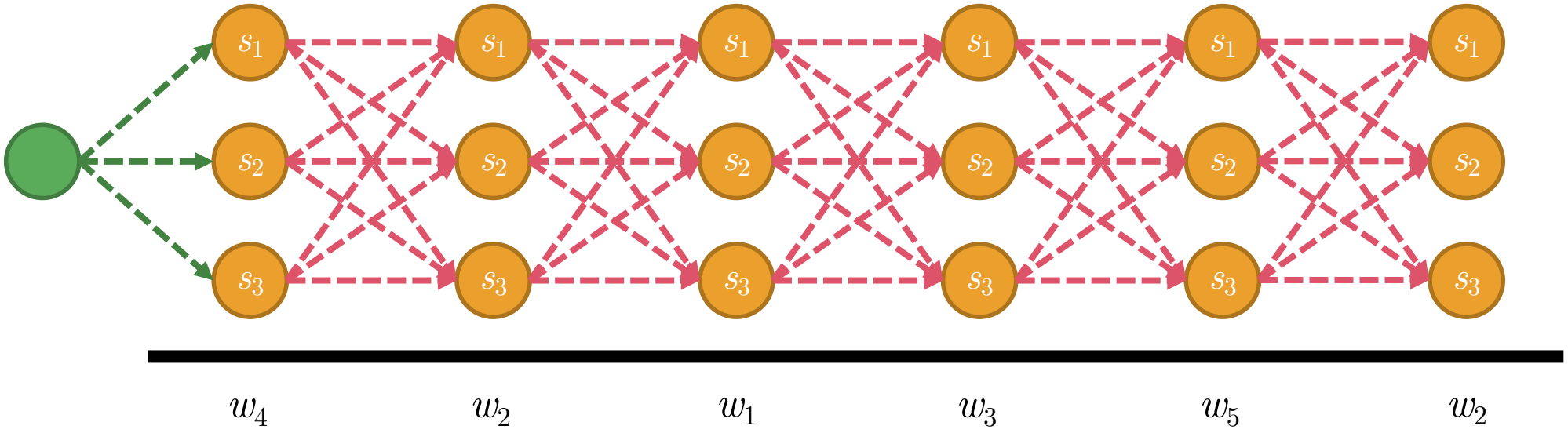
repeat:
  let p_σ, p_τ, p_e, q_σ, q_τ, q_e be empty hash tables and $q_\sigma = 0$
  for each string $\langle (w_1,s_1), \ldots, (w_N,s_N) \rangle$ in $\mathcal{D}$:
    compute forward expectation α
    compute backward expectation β
    for t in $[1,N-1]$:
      update $p_\tau[s_t,s_{t+1}]$ += $\alpha[s_t]^{(t)}\ \tau[s_t,s_{t+1}]\ e[s_{t+1},w_{t+1}]\ \beta[s_{t+1}]^{(t+1)}$
      update $q_\tau[s_t]$ += $\alpha[s_t]^{(t)}\ \tau[s_t,s_{t+1}]\ \beta[s_{t+1}]^{(t+1)}$
    for t in $[1,N]$:
      update $p_e[s_t,w_t]$ += $\alpha[s_t]^{(t)}\ e[s_t,w_t]\ \beta[s_t]^{(t)}$
      update $q_e[s_t]$ += $\alpha[s_t]^{(t)}\ e[s_t,w_t]\ \beta[s_t]^{(t)}$
    update $p_\sigma[s_t]$ += $\beta[s_t]^{(1)}$
    update q_σ += $\beta[s_t]^{(1)}$
  for each key (s_x,s_y) in τ: set $\tau[s_x,s_y] = (p_\tau[s_x,s_y] + \mu_\tau[s_x,s_y]) \ / \ (q_\tau[s_x] + \sum_y \mu_\tau[s_x,s_y])$
  for each key (s_x,w_i) in e: set $e[s_x,w_i] = (p_e[s_x,w_i] + \mu_e[s_x,w_i]) \ / \ (q_e[s_x] + \sum_i \mu_e[s_x,w_i])$
  for each key s_x in σ: set $\sigma[s_x] = (p_\sigma[s_x] + \mu_\sigma[s_x]) \ / \ (q_\sigma + \sum_x \mu_\sigma[s_x])$
until σ, τ, and e converge

Each μ_τ, μ_e, and μ_σ are priors for transition, emission, and starting, respectively

2. Inference with Viterbi Algorithm (Viterbi, 1967)

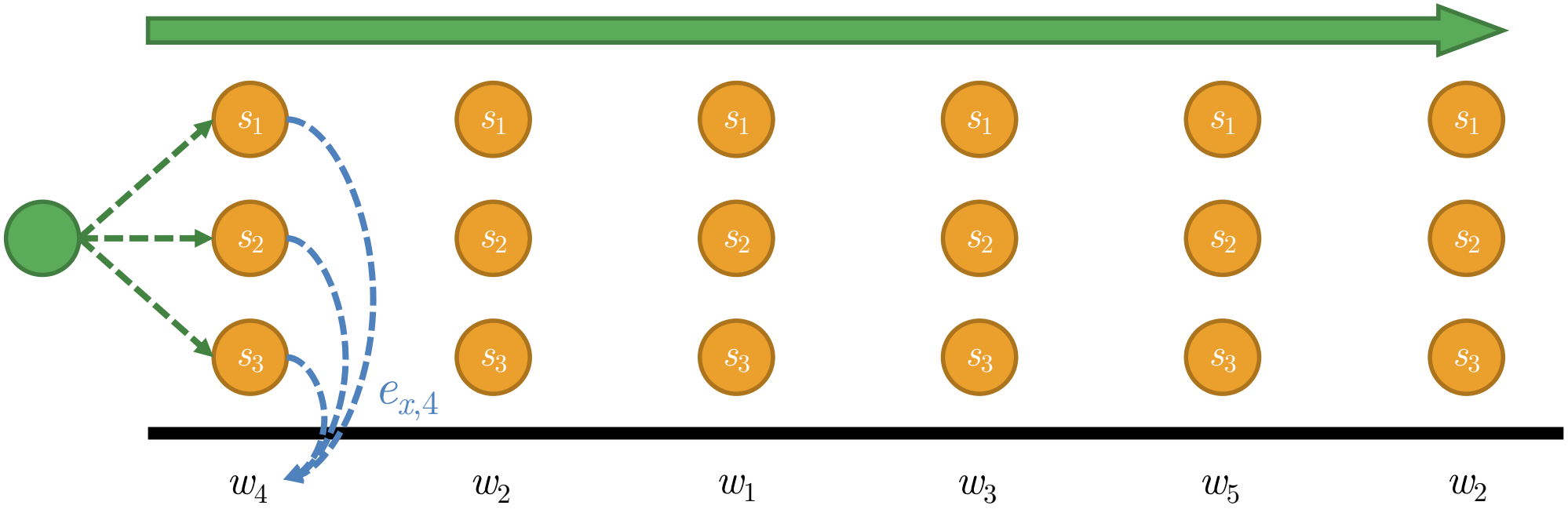
Viterbi Algorithm for HMMs

- Choose the most likely path based on the HMM's parameters
  - Step 1: Sweep from left to right to compute path scores
  - Step 2: Sweep from right to left to select the best path



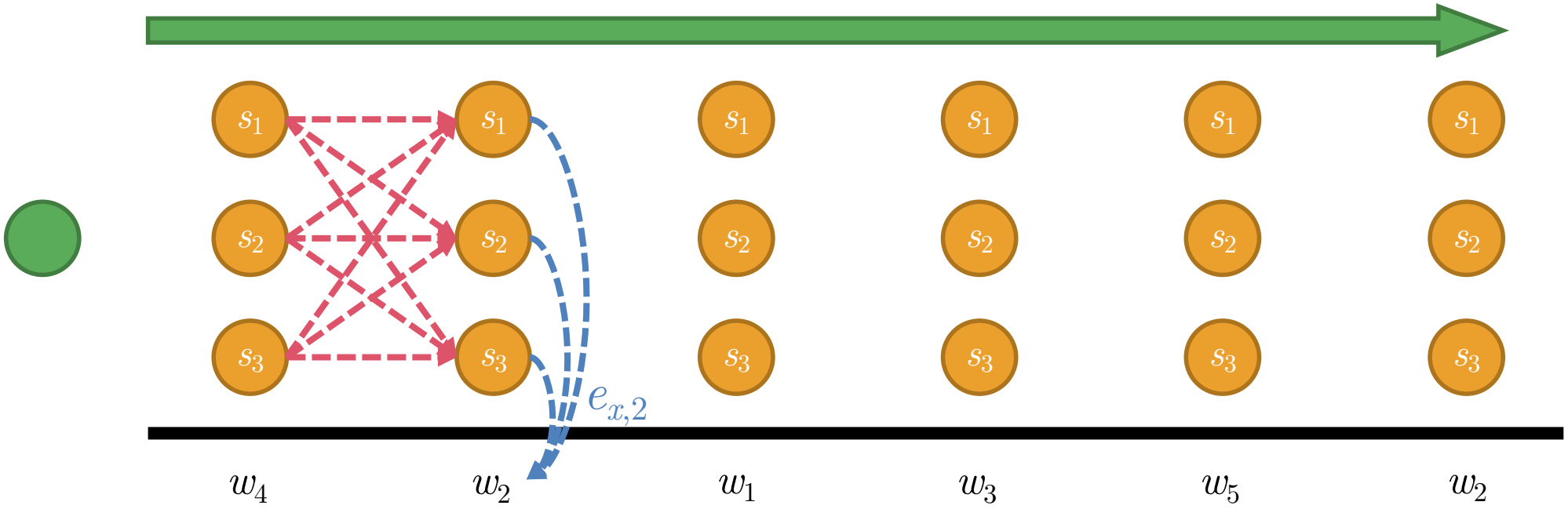
Step 1: Sweeping Left to Right

- At step $t = 1$ with observation w_i:
  - For each state s_x with observation w_i, set $p[s_x, 1] = \sigma_x \, e_{xi}$



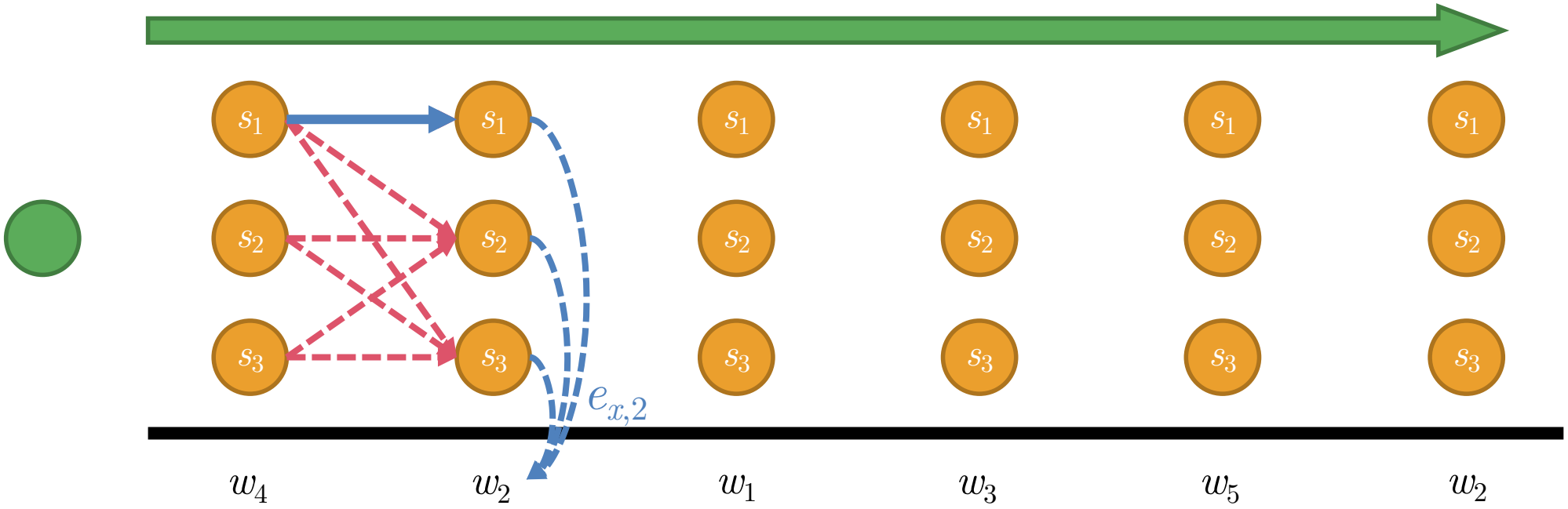
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



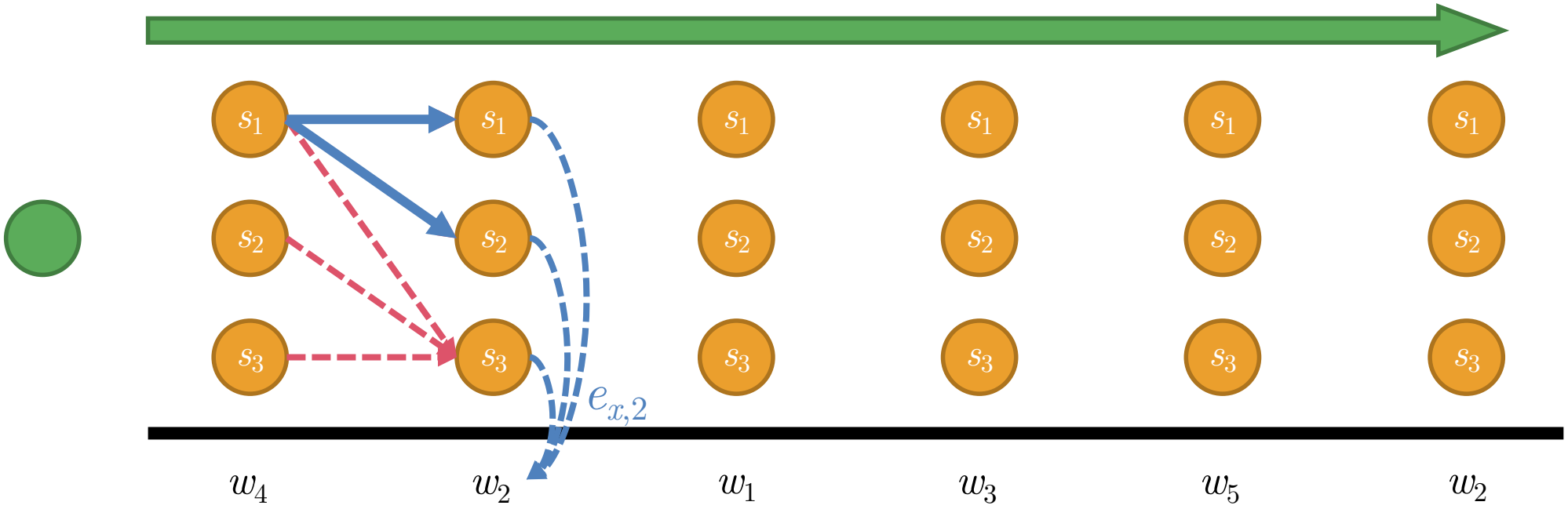
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



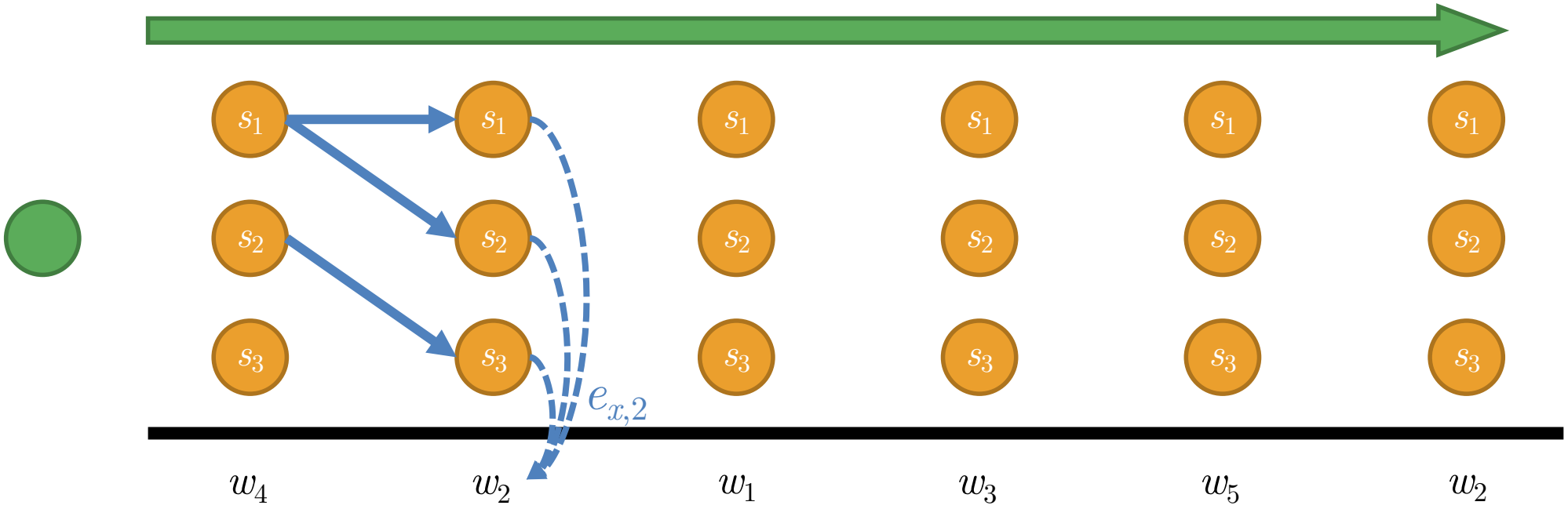
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



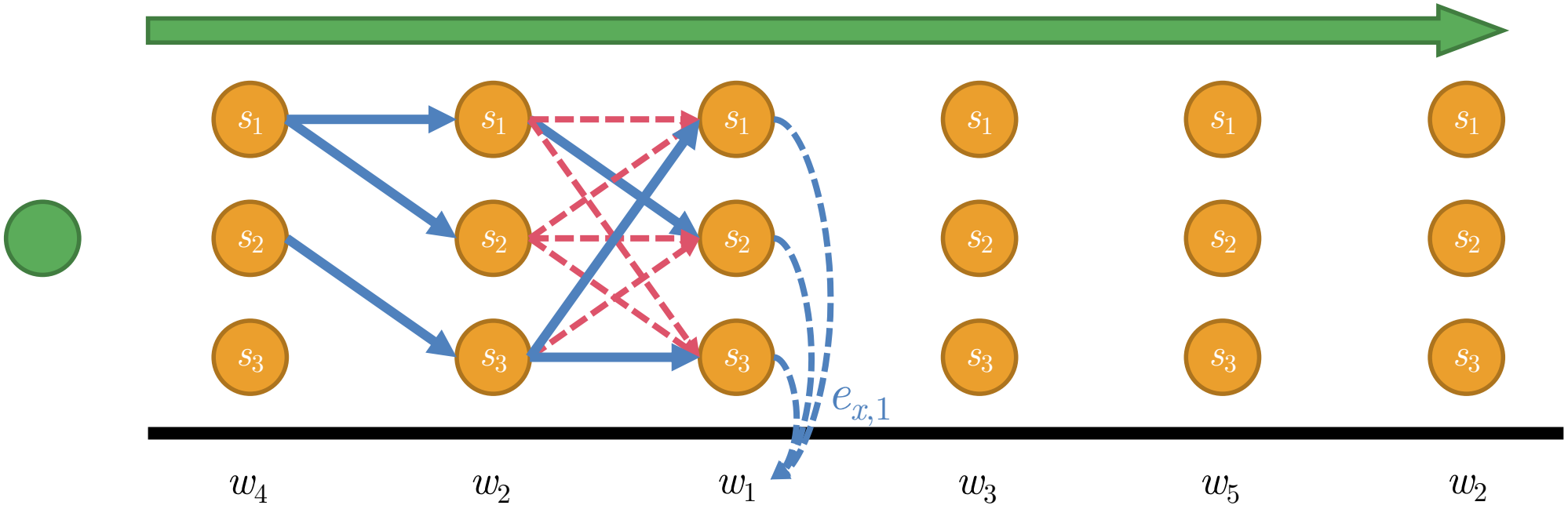
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



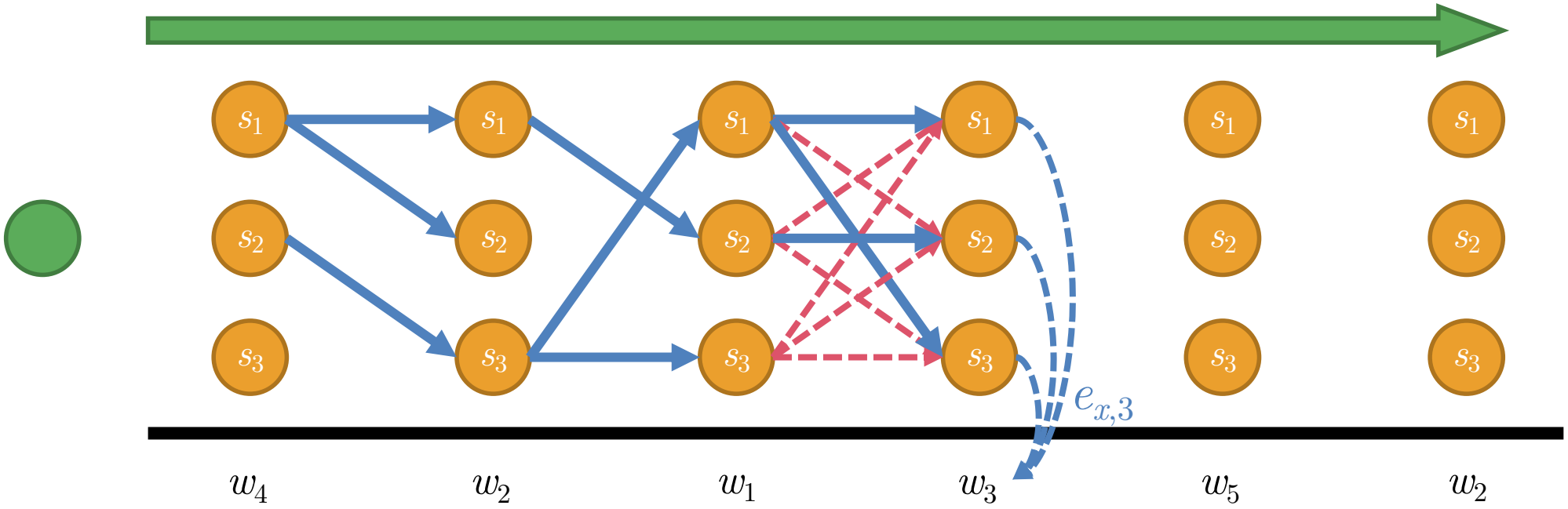
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



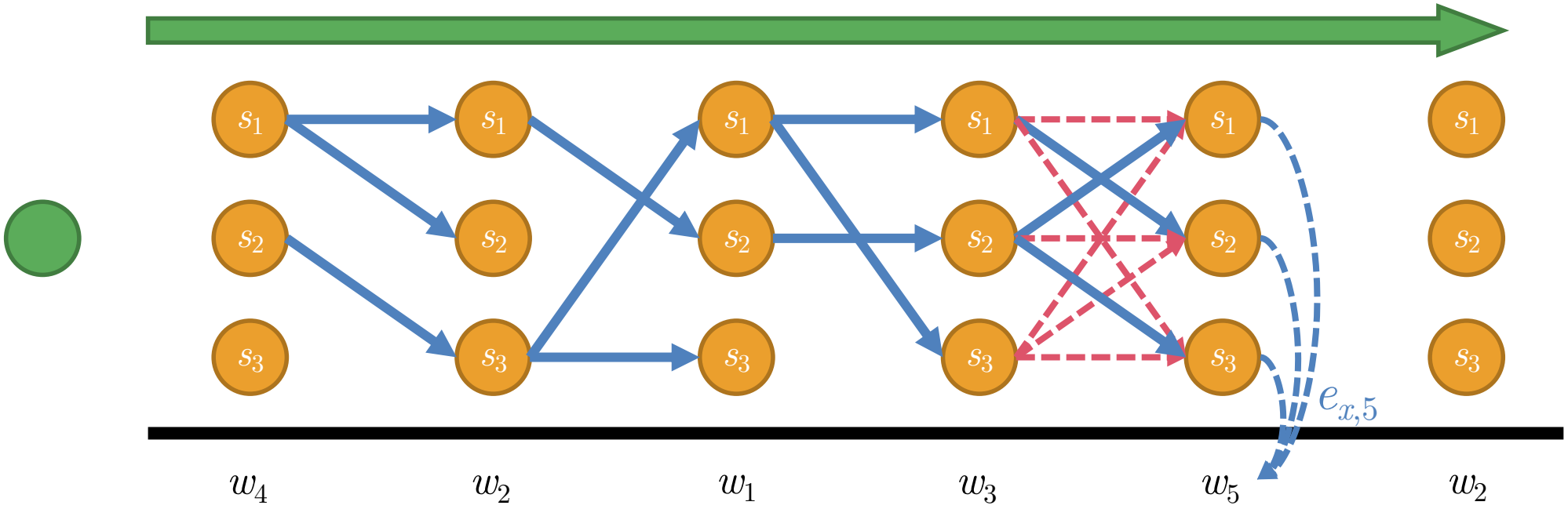
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



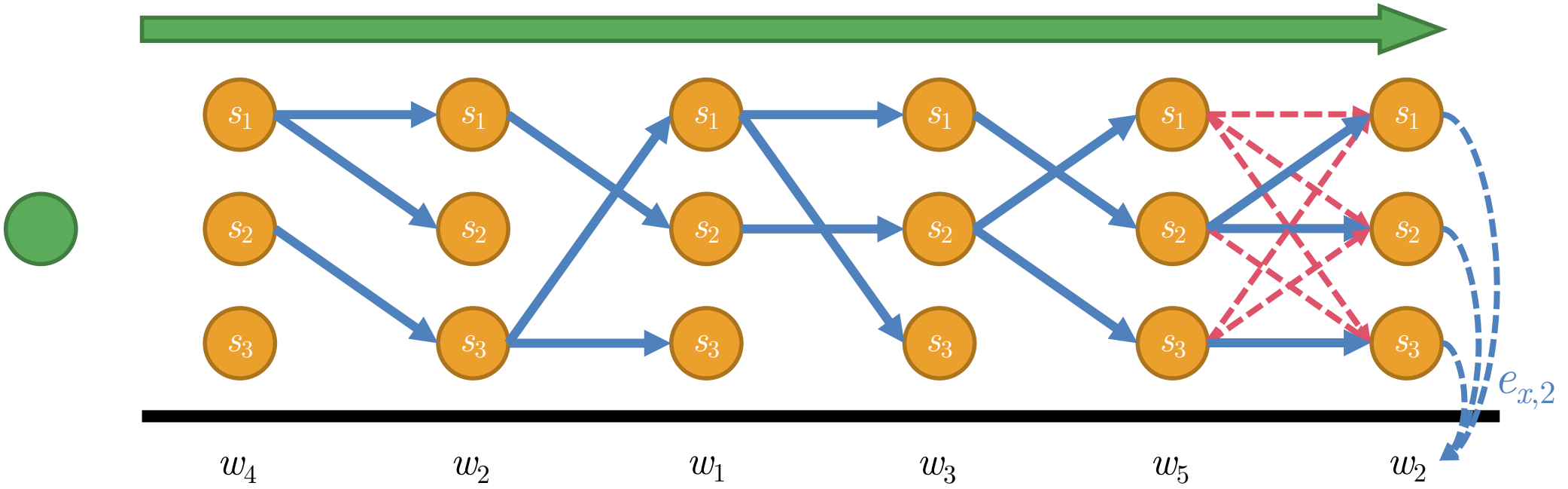
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



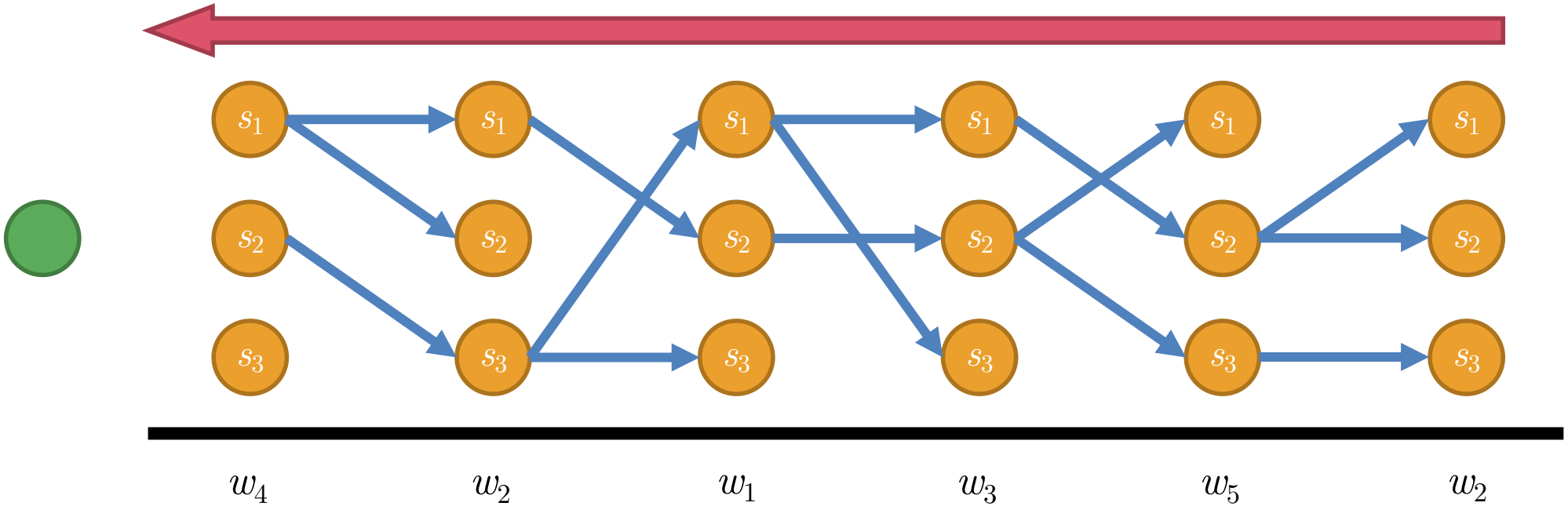
Step 1: Sweeping Left to Right

- At any step t with observation w_i:
  - For each state s_x, we set $p[x,t] = \arg\max_y p[y,t\text{-}1]\ \tau_{yx}\ e_{xi}$, and we establish a path from $(s_y,t\text{-}1)$ to (s_x,t)



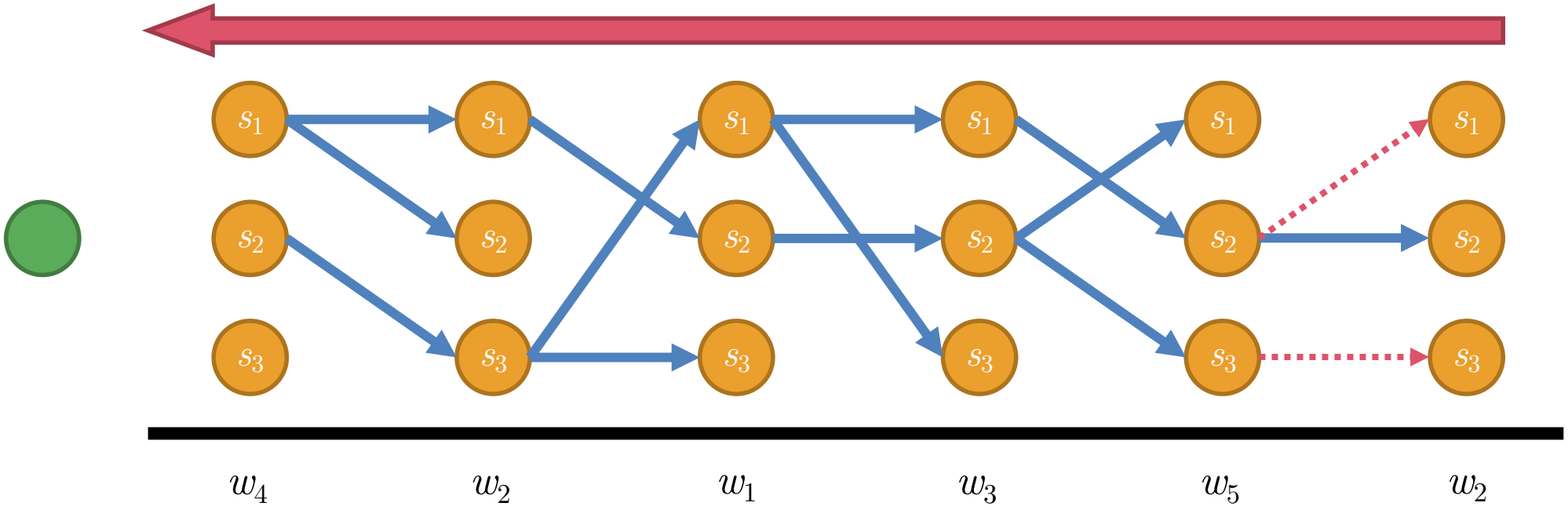
Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



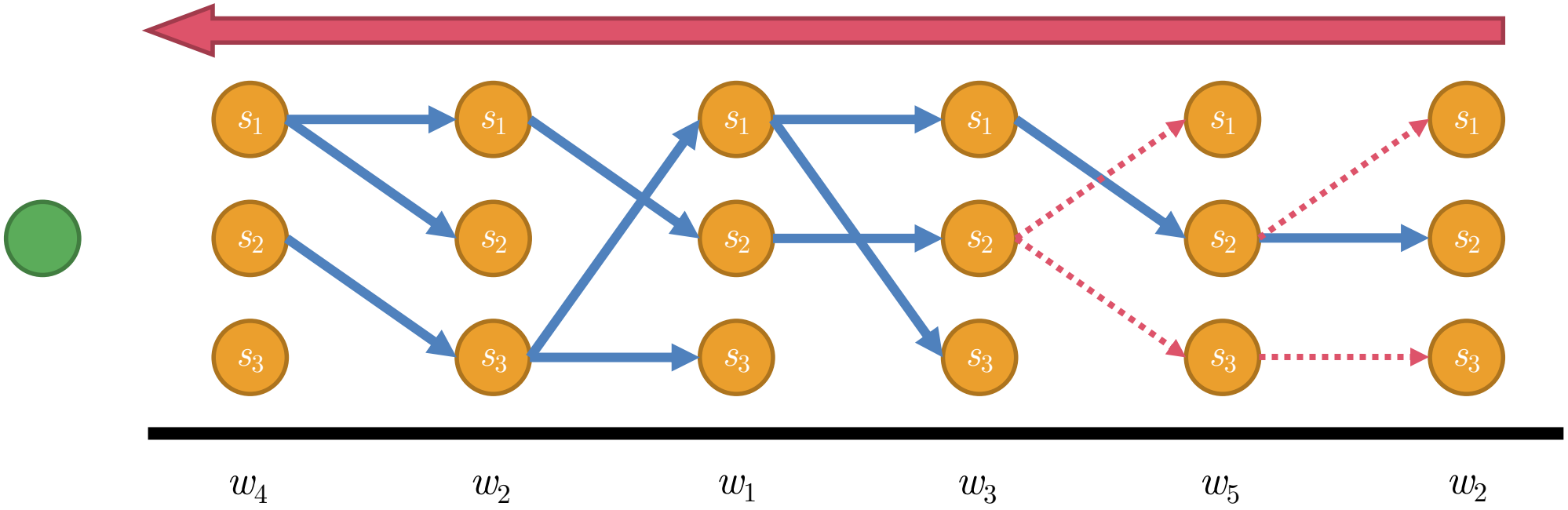
Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



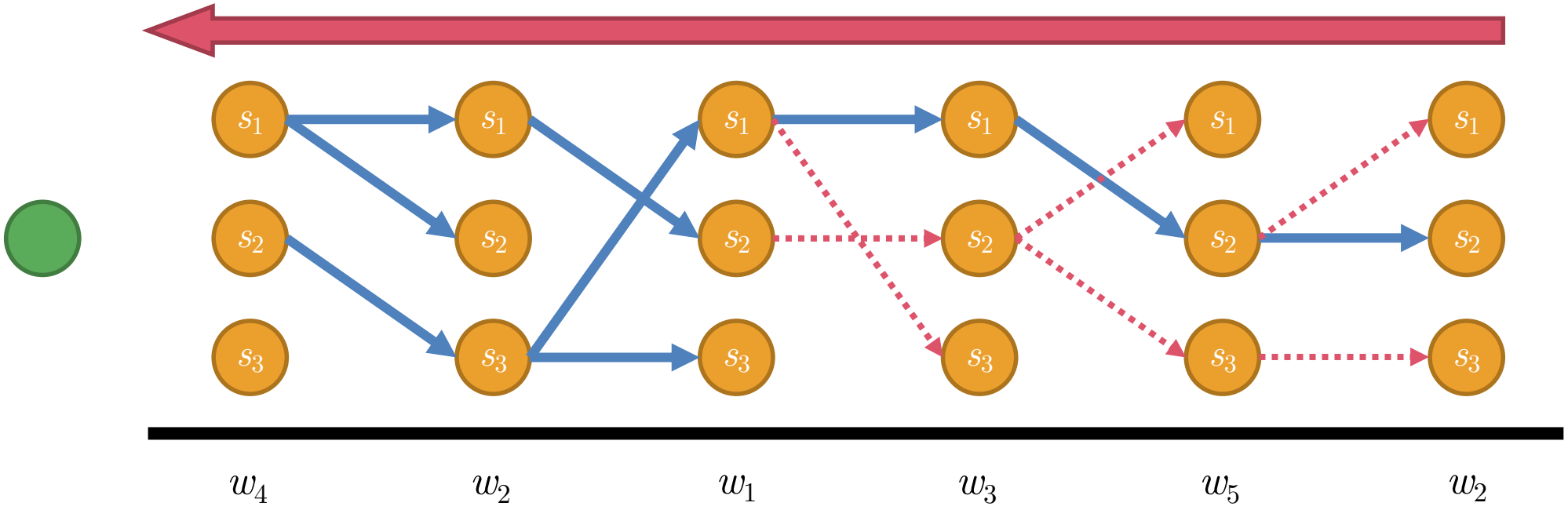
Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



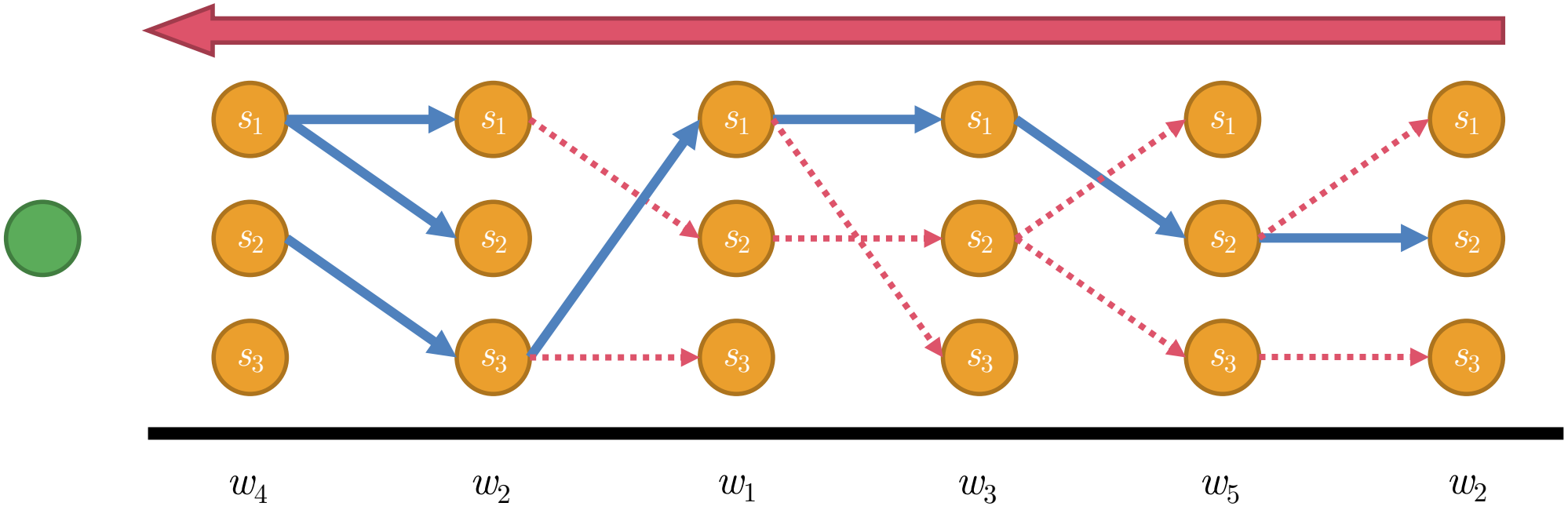
Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



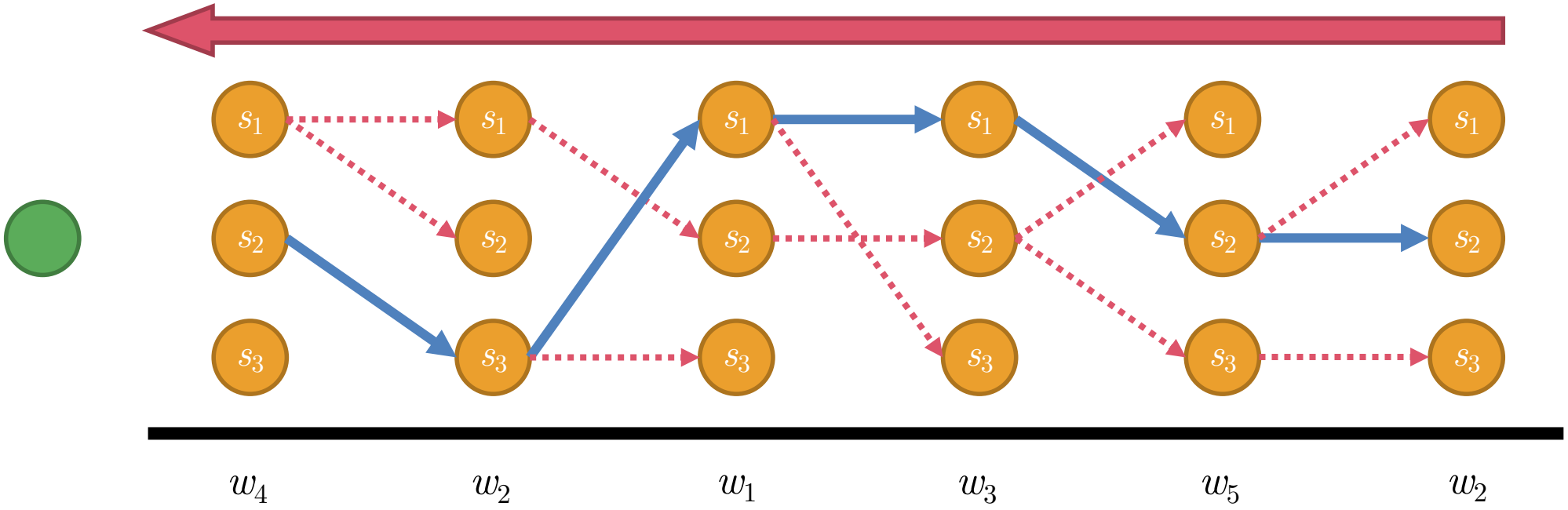
Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



Step 2: Sweeping from Right to Left

- At step $t = N$:
  - We choose the state s_x that has the most score $p[x]$ and trace back to step $t = 1$ to get the most likely sequence



Tree Prediction

Tree Prediction

- We assume that a given symbolic string is statistically generated by a hidden hierarchical structure

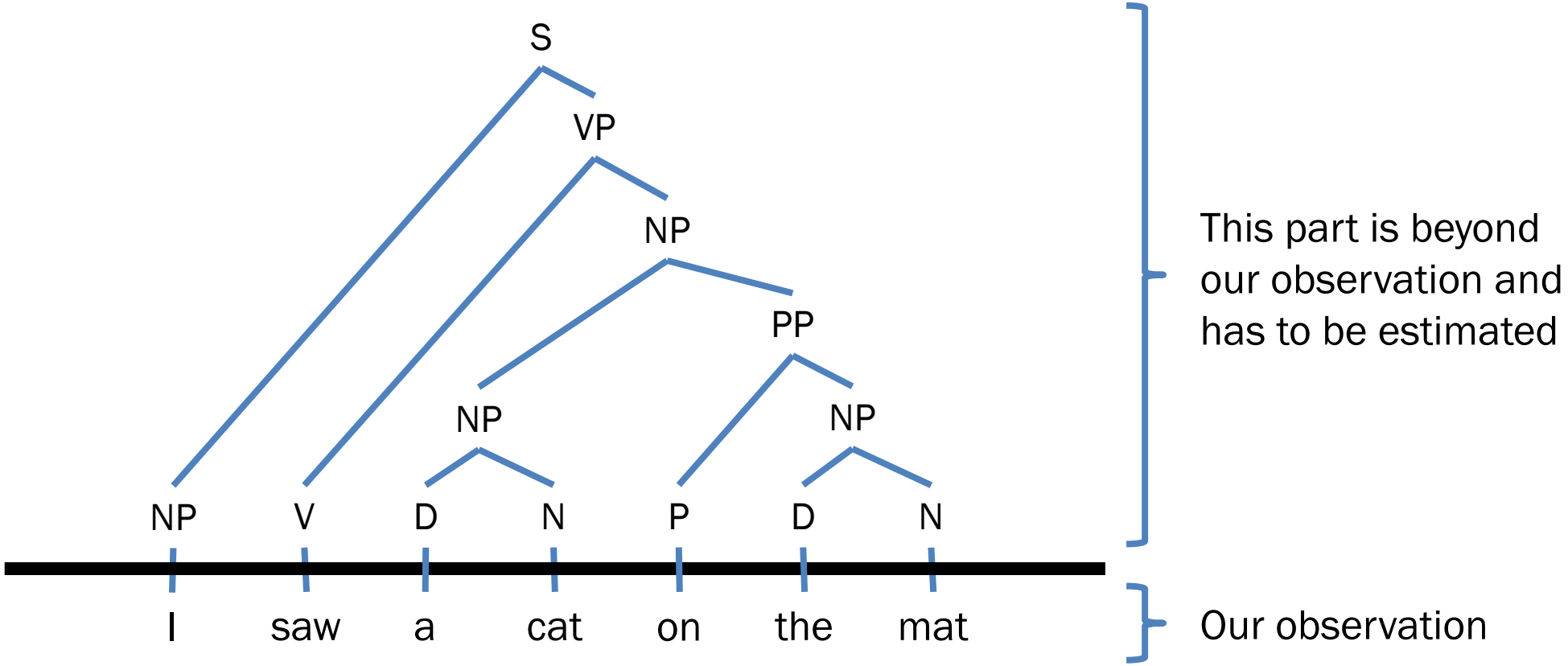
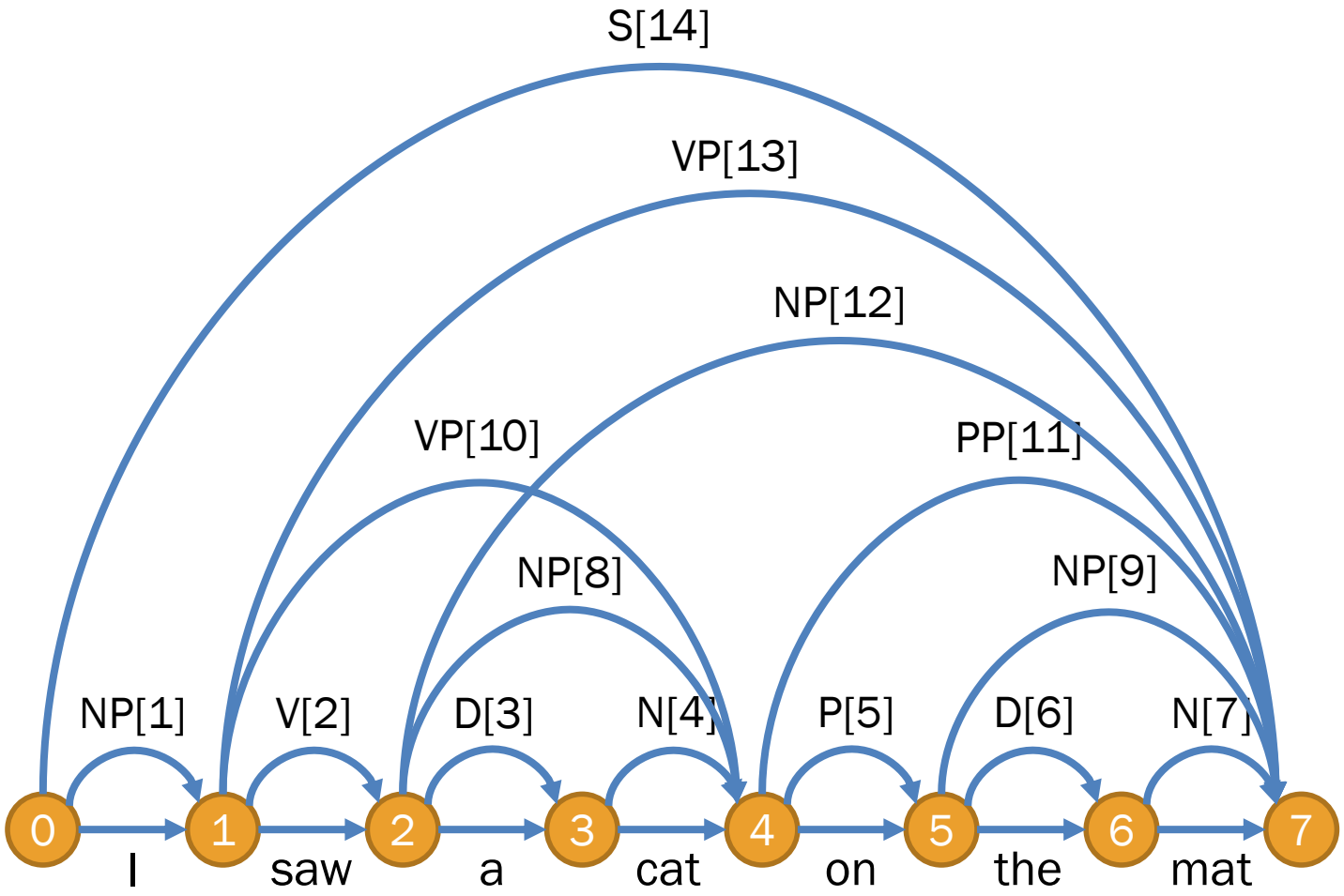


Chart Parsing

- E.g. Parsing “I saw a cat on the mat”



Item	Content
1	NP → I
2	V → saw
3	D → a
4	N → cat
5	P → on
6	D → the
7	N → mat

Item	Content
8	NP → [3, 4]
9	NP → [6, 7]
10	VP → [2, 8]
11	PP → [5, 9]
12	NP → [8, 11]
13	VP → [2, 12] [10, 11]
14	S → [1, 13]

Probabilistic Context-Free Grammar (PCFG)

- Each context-free rule is assigned with a probability value

Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Probabilistic Context-Free Grammar (PCFG)

- Assume that a string has been parsed with CKY Algorithm into a packed chart
- We will choose the most likely tree, where:

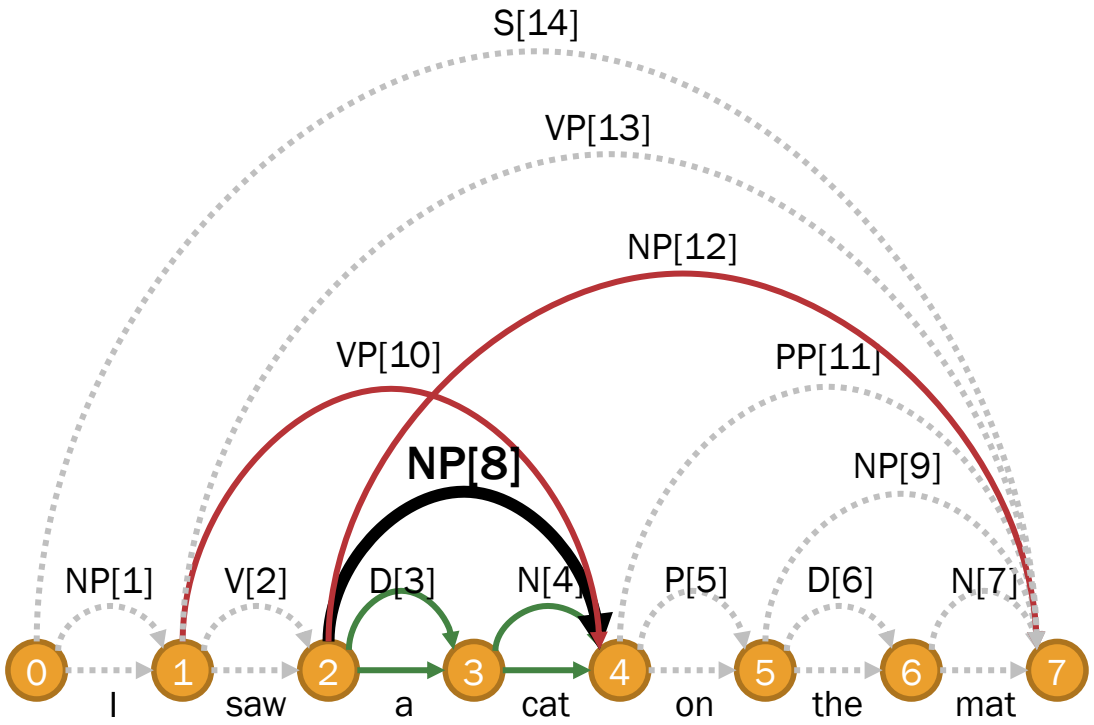
$$P\Big(\ \begin{array}{ccc} & \text{VP} & \\ \text{V} & & \text{NP} \\ & & \text{DT}\quad\text{NN} \\ \textbf{saw} & & \textbf{a}\quad\textbf{cat} \end{array}\ \Big) = \begin{array}{l} p(\text{VP} \rightarrow \text{V NP}) \\ \times\ p(\text{V} \rightarrow \textbf{saw}) \\ \times\ p(\text{NP} \rightarrow \text{D N}) \\ \times\ p(\text{D} \rightarrow \textbf{a}) \\ \times\ p(\text{N} \rightarrow \textbf{cat}) \end{array}$$

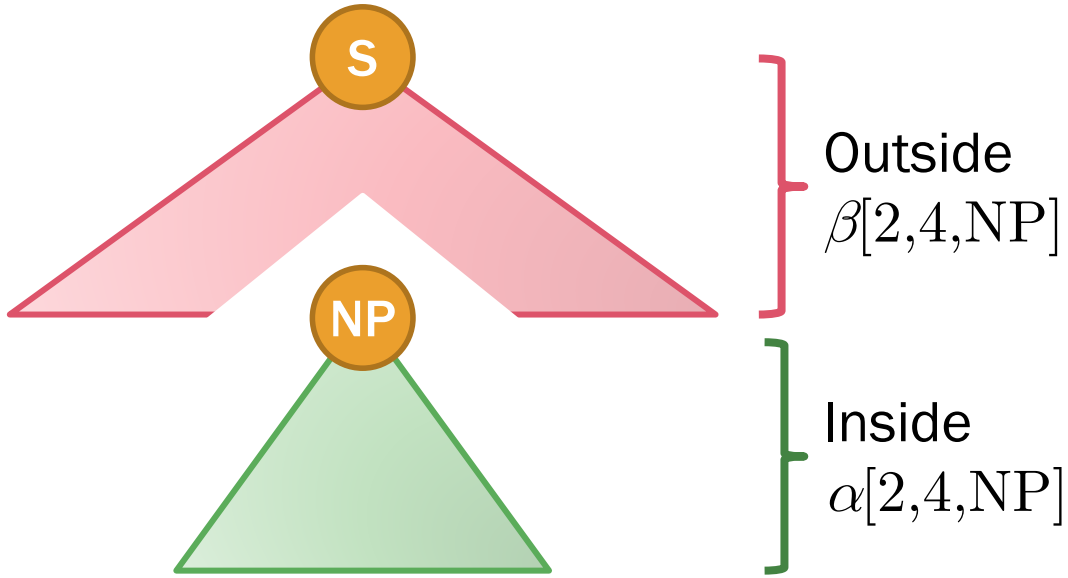
1. Parameter Estimation with EM

(Baker, 1979; Lari and Young, 1990)

Q Function: Node Expectations

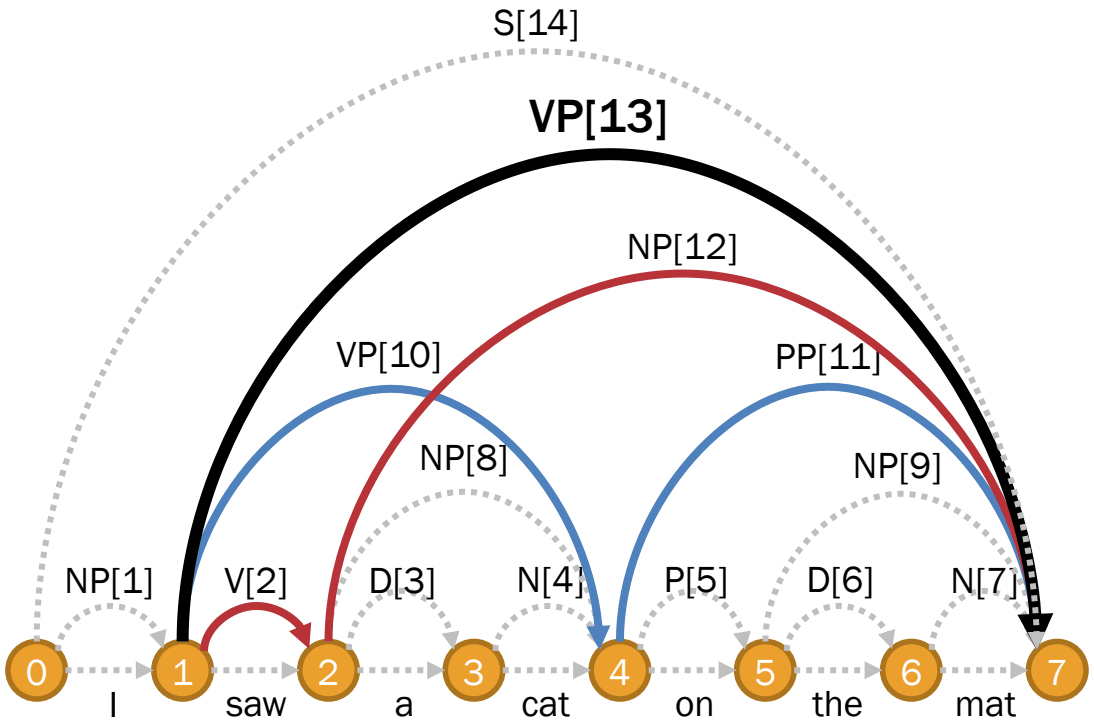
- For any item in the packed chart, there are 2 expectations
  - **Inside expectation** α_{xyi} with span (x,y) and category c_i
  - **Outside expectation** β_{xyi} with span (x,y) and category c_i





Q Function: Node Expectations

- Letting $r_{ijk} = p(c_i \rightarrow c_j c_k)$, we recursively define
  - **Inside expectation** $\alpha_{xyi} = \sum_{zjk} r_{ijk}\, \alpha_{xzj}\, \alpha_{zyk}$
  - **Outside expectation** $\beta_{xyi} = \sum_{zjk} r_{jik}\, \alpha_{yzk}\, \beta_{xzj} + \sum_{zjk} r_{jki}\, \alpha_{xzk}\, \beta_{zyj}$

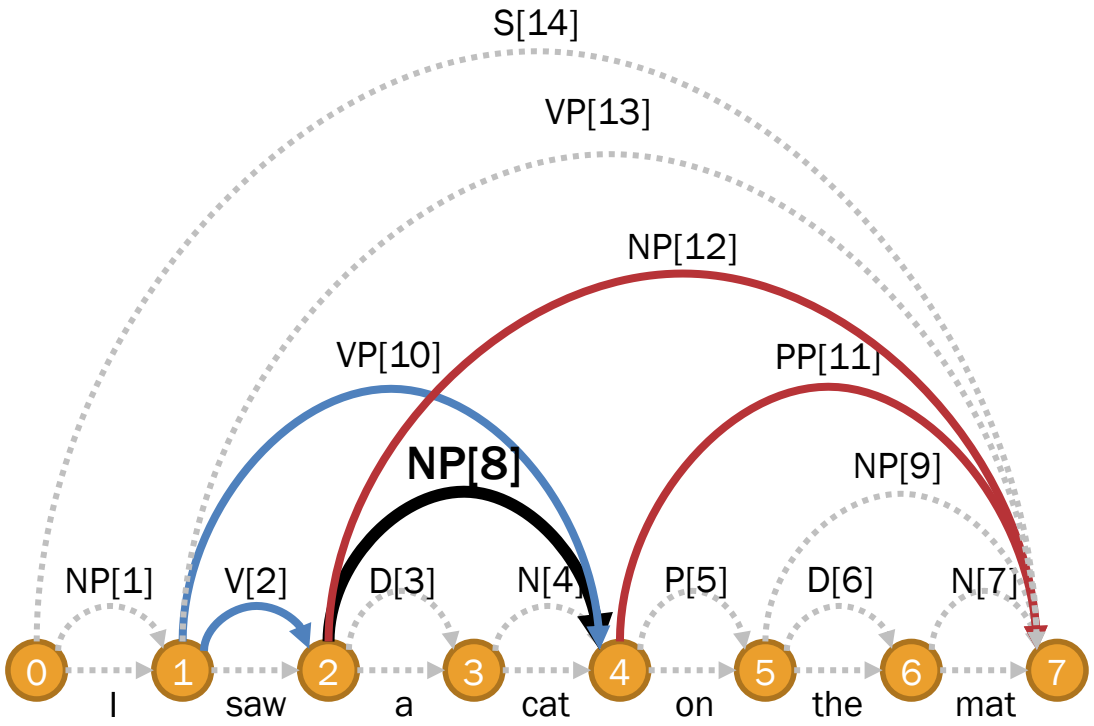


Example: inside expectation

$$\alpha[1,7,\text{VP}] = p(\text{VP} \rightarrow \text{V NP})\ \alpha[1,2,\text{V}]\ \alpha[2,7,\text{NP}] + p(\text{VP} \rightarrow \text{VP PP})\ \alpha[1,4,\text{VP}]\ \alpha[4,7,\text{PP}]$$

Q Function: Node Expectations

- Letting $r_{ijk} = p(c_i \rightarrow c_j c_k)$, we recursively define
  - **Inside expectation** $\alpha_{xyi} = \sum_{zjk} r_{ijk}\, \alpha_{xzj}\, \alpha_{zyk}$
  - **Outside expectation** $\beta_{xyi} = \sum_{zjk} r_{jik}\, \alpha_{yzk}\, \beta_{xzj} + \sum_{zjk} r_{jki}\, \alpha_{xzk}\, \beta_{zyj}$

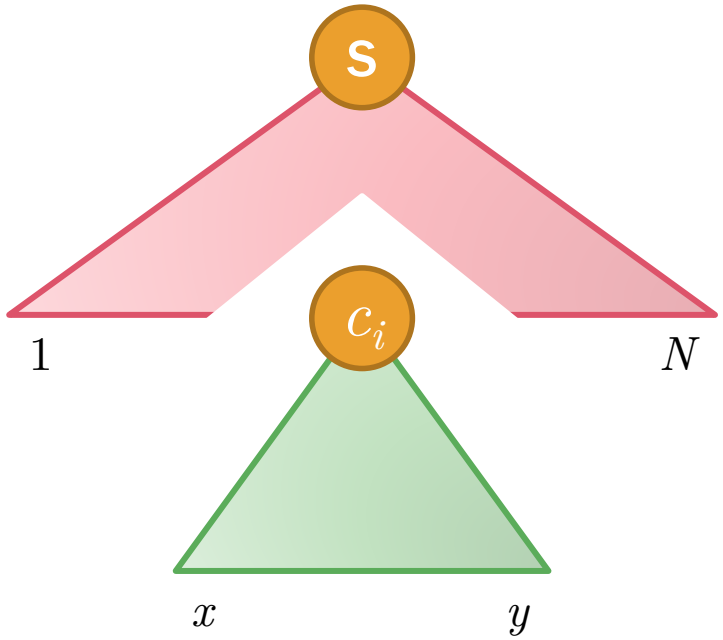


Example: outside expectation

$$\beta[2,4,\text{NP}] = p(\text{VP}\rightarrow\text{V NP})\ \alpha[1,2,\text{V}]\ \beta[1,4,\text{VP}] + p(\text{NP}\rightarrow\text{NP PP})\ \alpha[4,7,\text{PP}]\ \beta[2,7,\text{NP}]$$

Q Function: Node Expectations

- **Inside expectation**
  - Basic case: $\alpha_{x,x+1,i} = p(c_i \rightarrow w_t)$ [observation o_x = word w_t]
  - Recursive case: $\alpha_{xyi} = \sum_{zjk} r_{ijk}\, \alpha_{xzj}\, \alpha_{zyk}$

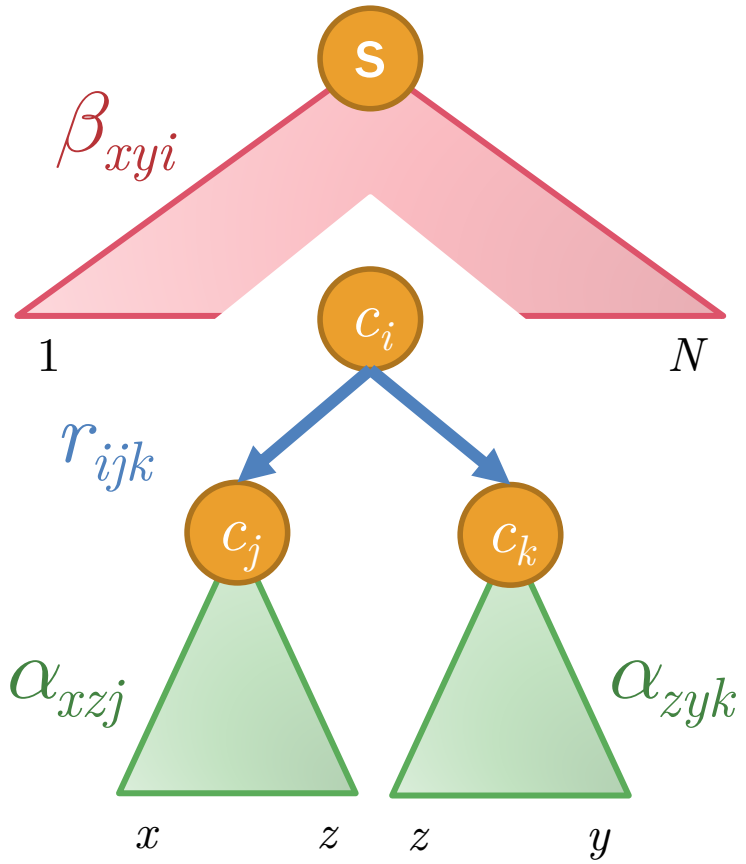


- **Outside expectation**
  - Basic case: $\beta_{1,N,\mathrm{S}} = 1$ [start symbol]
  - Recursive case: $\beta_{xyi} = \sum_{zjk} r_{jik}\, \alpha_{yzk}\, \beta_{xzj} + \sum_{zjk} r_{jki}\, \alpha_{xzk}\, \beta_{zyj}$

E-Step: Expectation of Branching

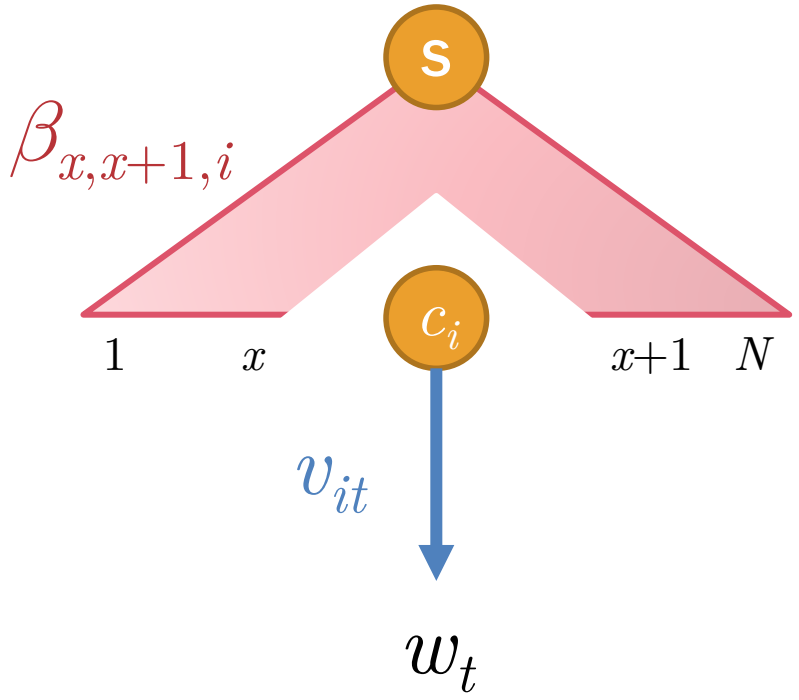
- Multiplication of β_{xyi} (outside score of c_i spanning $(1,N)$), r_{ijk} (probability of rule $c_i \rightarrow c_j c_k$), and α_{xzj} and α_{zyk} (inside score of both daughters)

$$\zeta_{xzyijk} = \beta_{xyi}\, r_{ijk}\, \alpha_{xzj}\, \alpha_{zyk}$$



E-Step: Expectation of Lexical Items

- Multiplication of $\beta_{x,x+1,i}$ (outside score of c_i spanning $(x,x+1)$), and v_{it} (probability of lexical item $c_i \rightarrow w_t$)



$$\xi_{xit} = \beta_{x,x+1,i}\, v_{it}$$

M-Step: Update Equations

- Update the parameters with the following equations
  - Branching

$$\hat{r}_{ijk} = p(c_i \rightarrow c_j c_k) = \frac{\sum_\tau \sum_{(x,z,y)\in\tau} \zeta_{xzyijk}}{\sum_\tau \sum_{jk} \sum_{(x,z,y)\in\tau} \zeta_{xzyijk}}$$

  - Lexical item

$$\hat{v}_{it} = p(c_i \rightarrow w_t) = \frac{\sum_\tau \sum_{(x,t)\in\tau} \xi_{xit}}{\sum_\tau \sum_t \sum_{(x,t)\in\tau} \xi_{xit}}$$

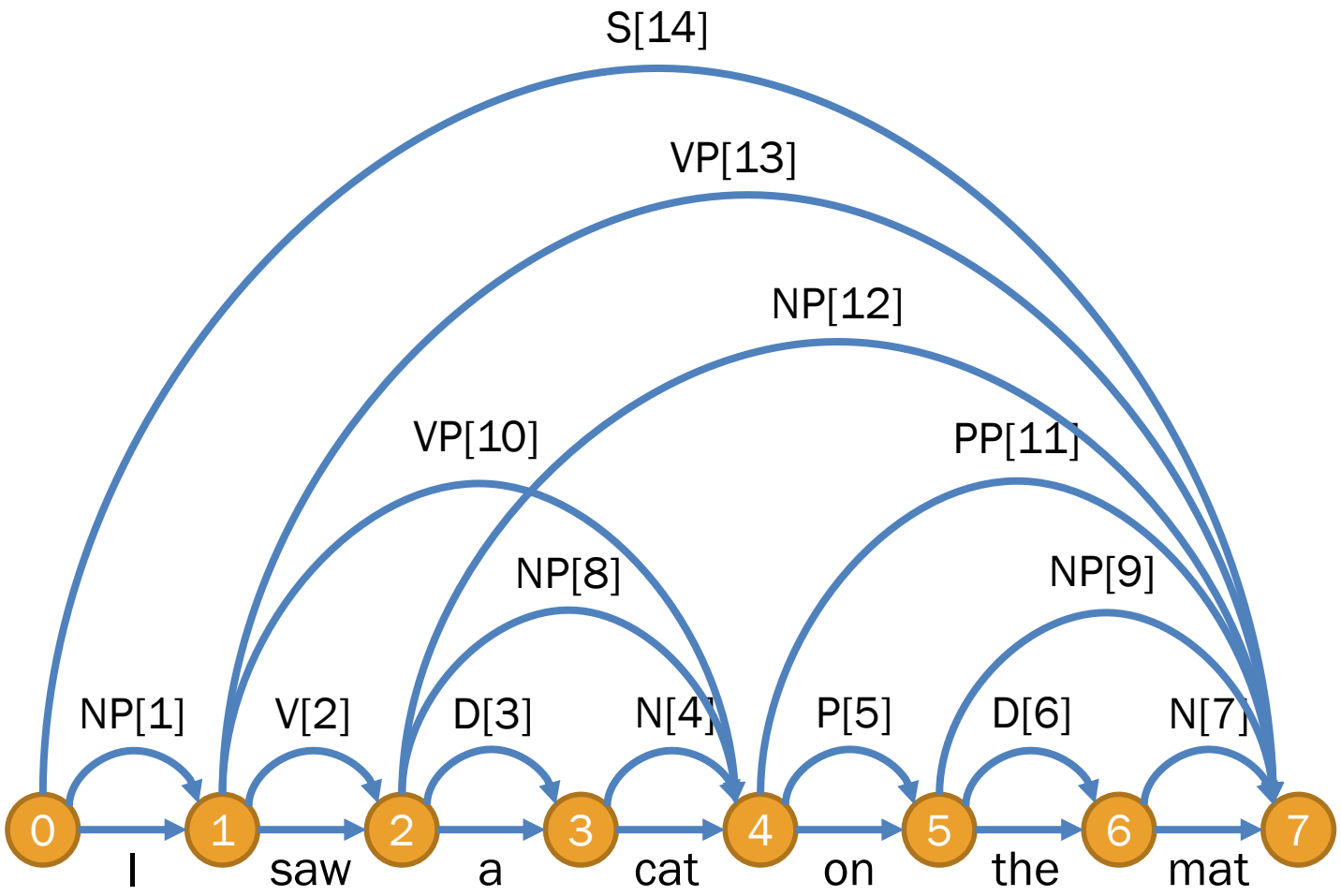
Inside-Outside Algorithm

```
repeat:
    let p_r, p_v, q be empty hash tables
    for each tree τ in 𝒟:
        compute inside expectation α from τ
        compute outside expectation β from τ
        compute expectation of branching φ from τ
        compute expectation of lexical item ψ from τ
        for each node (x,y,i) in τ:
            for each branching ⟨ (x,z,j), (z,y,k) ⟩ in (x,y,i):
                update p_r[i,j,k] += φ[x,z,y,i,j,k]   # expectation of branching
                update q[i] += φ[x,z,y,i,j,k]
            for each lexical item w_t:
                update p_v[i,t] += ψ[x,i,t]           # expectation of lexical item
                update q[i] += ψ[x,i,t]
    for each key (i,j,k) in r: set r[i,j,k] = p_r[i,j,k] / q[i]   # maximizing r
    for each key (i,t) in v: set v[i,t] = p_v[i,t] / q[i]         # maximizing v
until r and v converges
```

2. Inference with Viterbi Algorithm (Viterbi, 1967)

Viterbi Algorithm

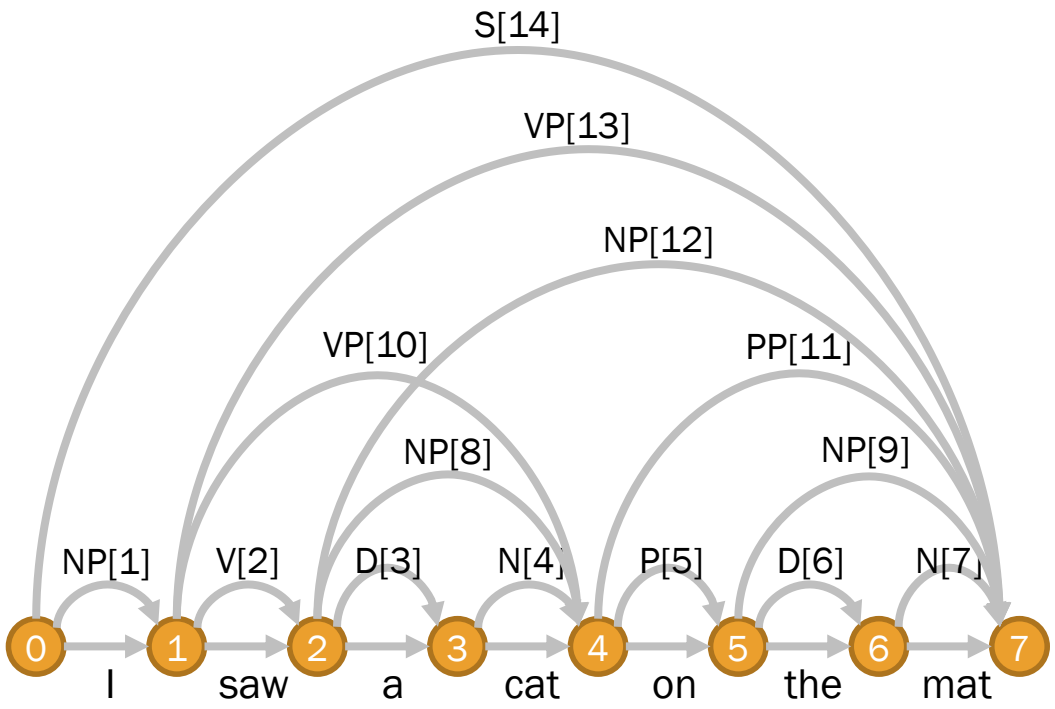
- E.g. Parsing “I saw a cat on the mat”



- Choosing the most likely tree from the packed chart
  - Based on the probability distribution of grammar rules
  - Using bottom-up dynamic programming
- Here, we will extract the tree from the item table

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	
2	V → saw	
3	D → a	
4	N → cat	
5	P → on	
6	D → the	
7	N → mat	

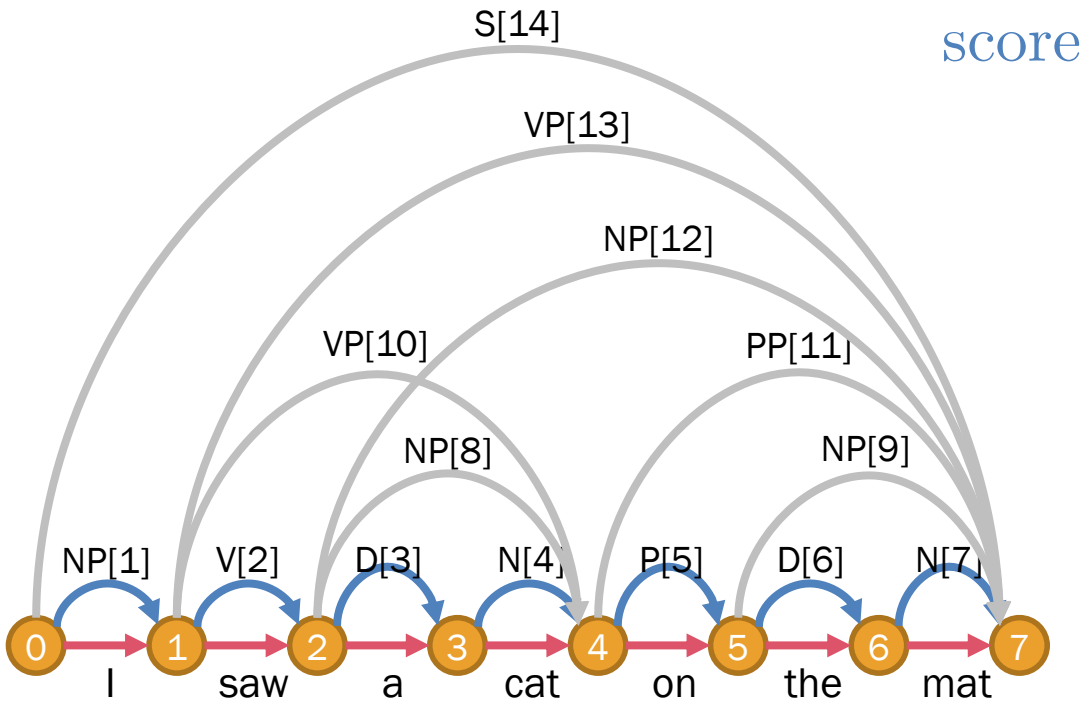
Item	Content	Score
8	NP → [3, 4]	
9	NP → [6, 7]	
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow w$

$\text{score}[m] = P(A \rightarrow w)$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	
2	V → saw	
3	D → a	
4	N → cat	
5	P → on	
6	D → the	
7	N → mat	

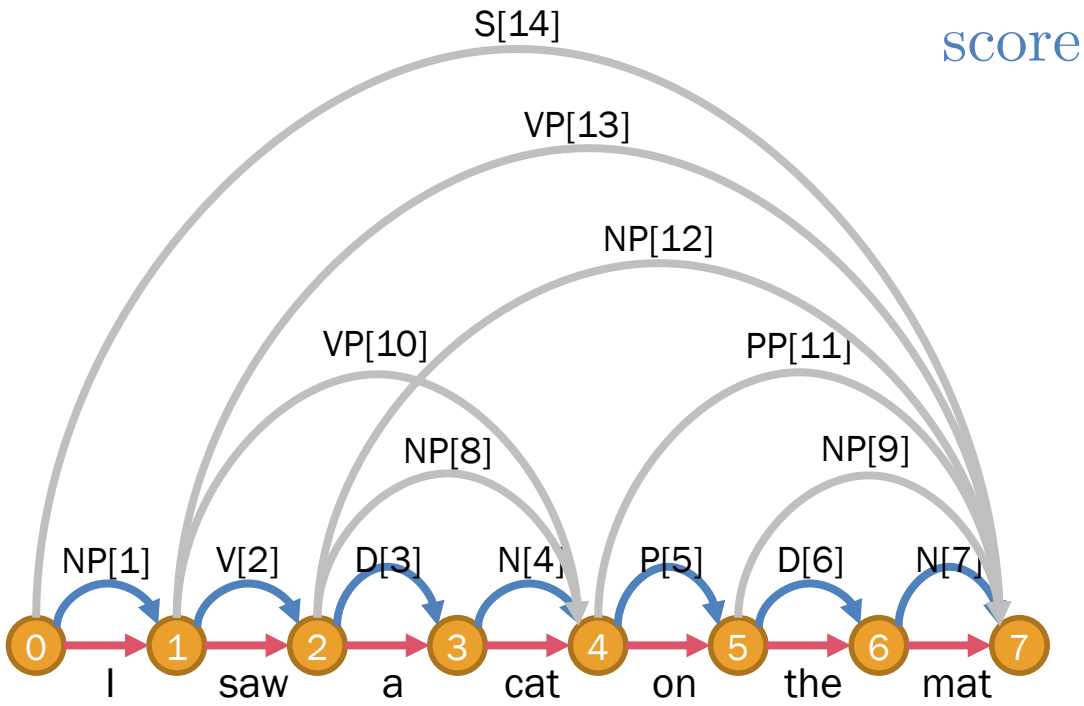
Item	Content	Score
8	NP → [3, 4]	
9	NP → [6, 7]	
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow w$

$\text{score}[m] = P(A \rightarrow w)$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

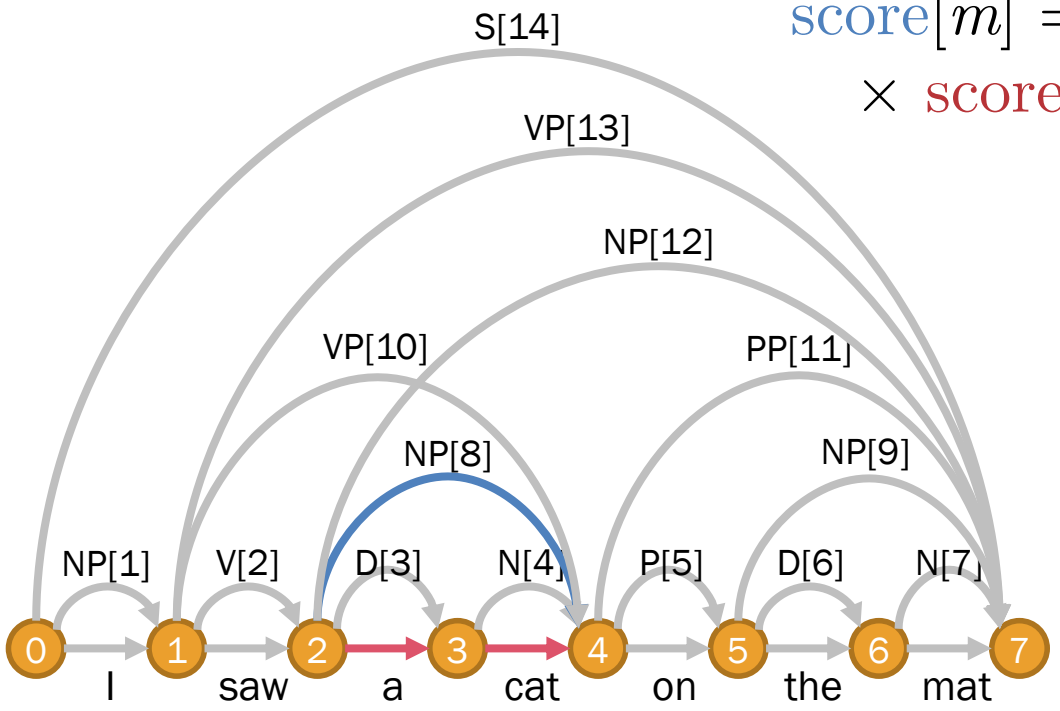
Item	Content	Score
8	NP → [3, 4]	
9	NP → [6, 7]	
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[m] = P(A \rightarrow B\ C) \times \text{score}[d_1] \times \text{score}[d_2]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

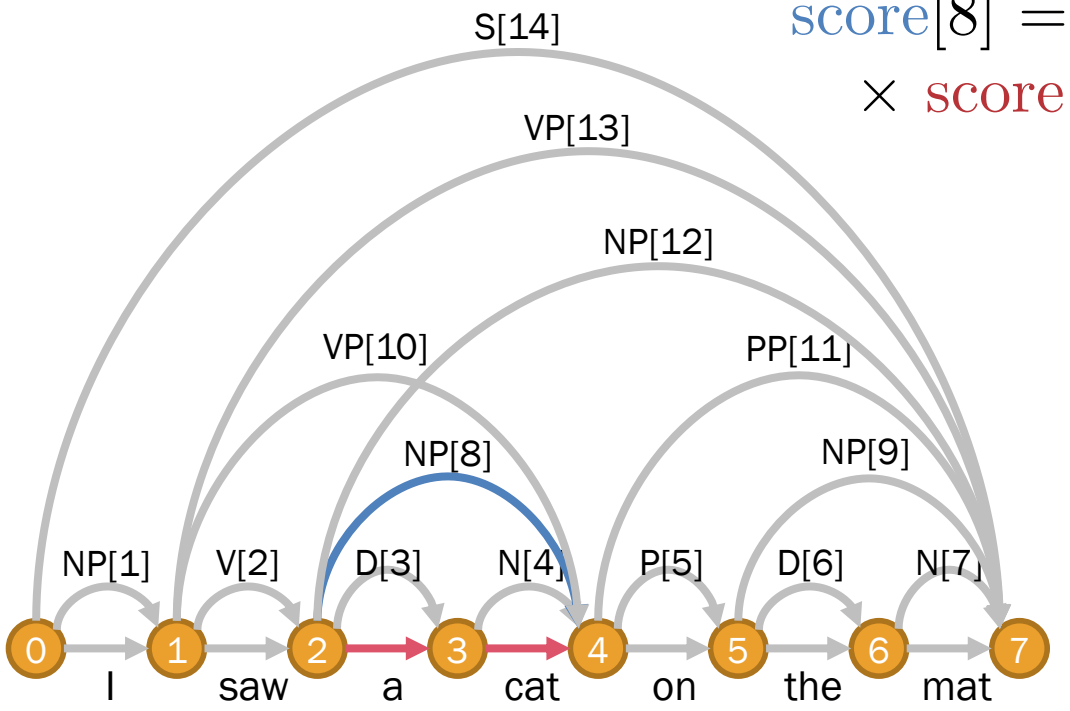
Item	Content	Score
8	NP → [3, 4]	
9	NP → [6, 7]	
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[8] = P(\text{NP} \rightarrow \text{D N}) \times \text{score}[3] \times \text{score}[4]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

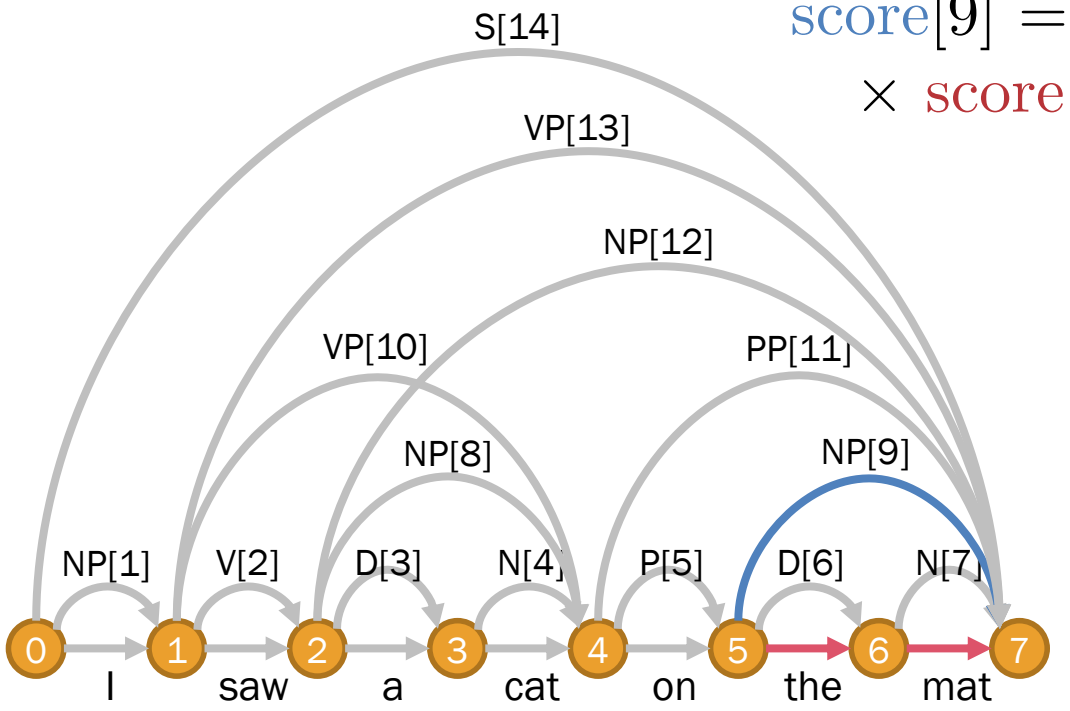
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[9] = P(\text{NP} \rightarrow \text{D N}) \times \text{score}[6] \times \text{score}[7]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

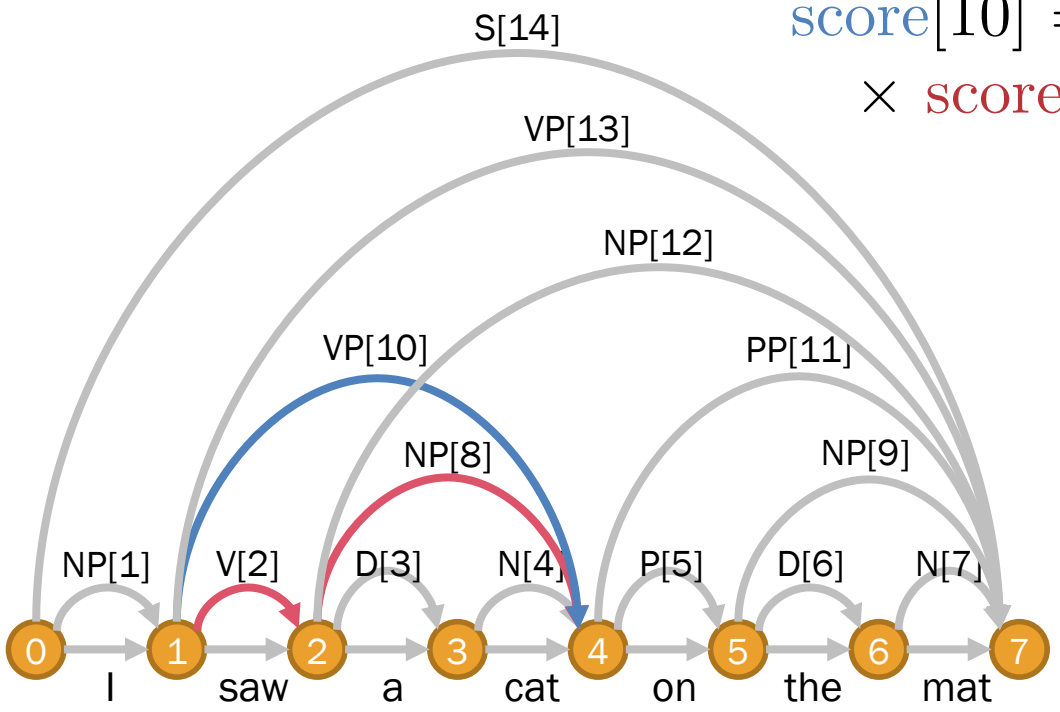
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[10] = P(\text{VP} \rightarrow \text{V NP}) \times \text{score}[2] \times \text{score}[8]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

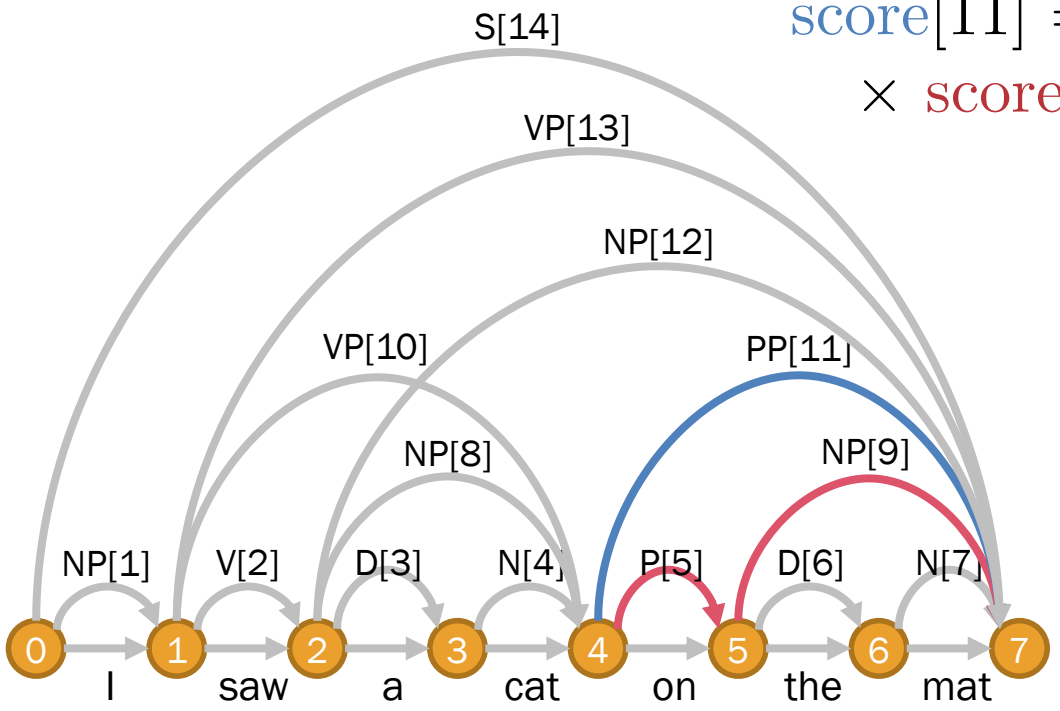
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[11] = P(\text{PP} \rightarrow \text{P NP}) \times \text{score}[5] \times \text{score}[9]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

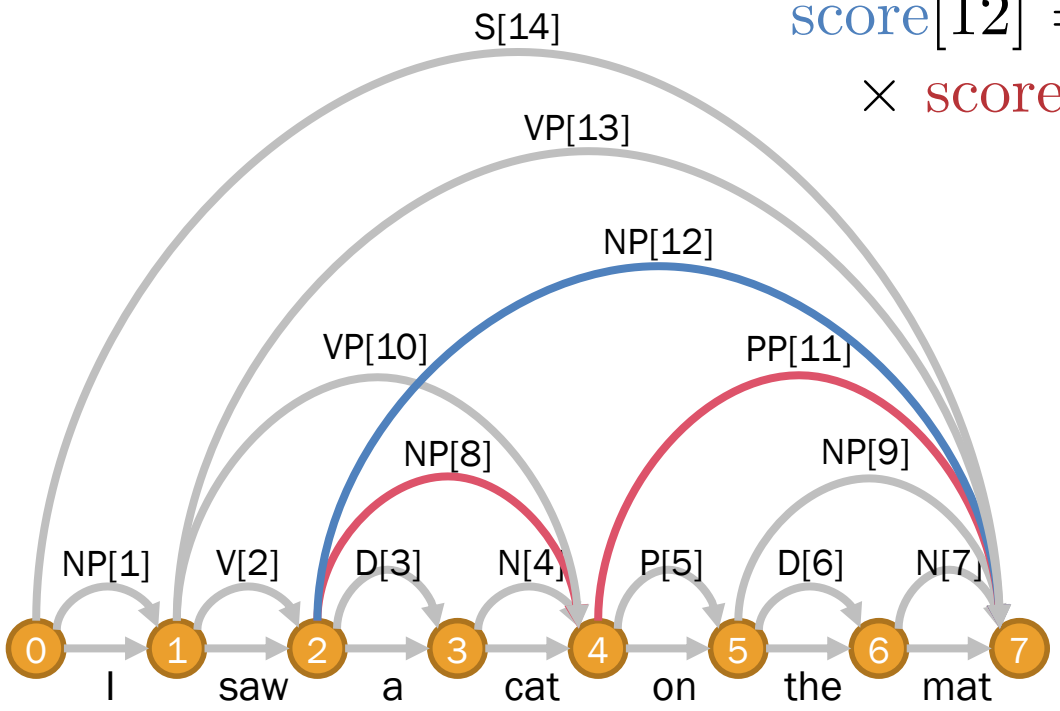
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[12] = P(\text{NP} \rightarrow \text{NP PP}) \times \text{score}[8] \times \text{score}[11]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

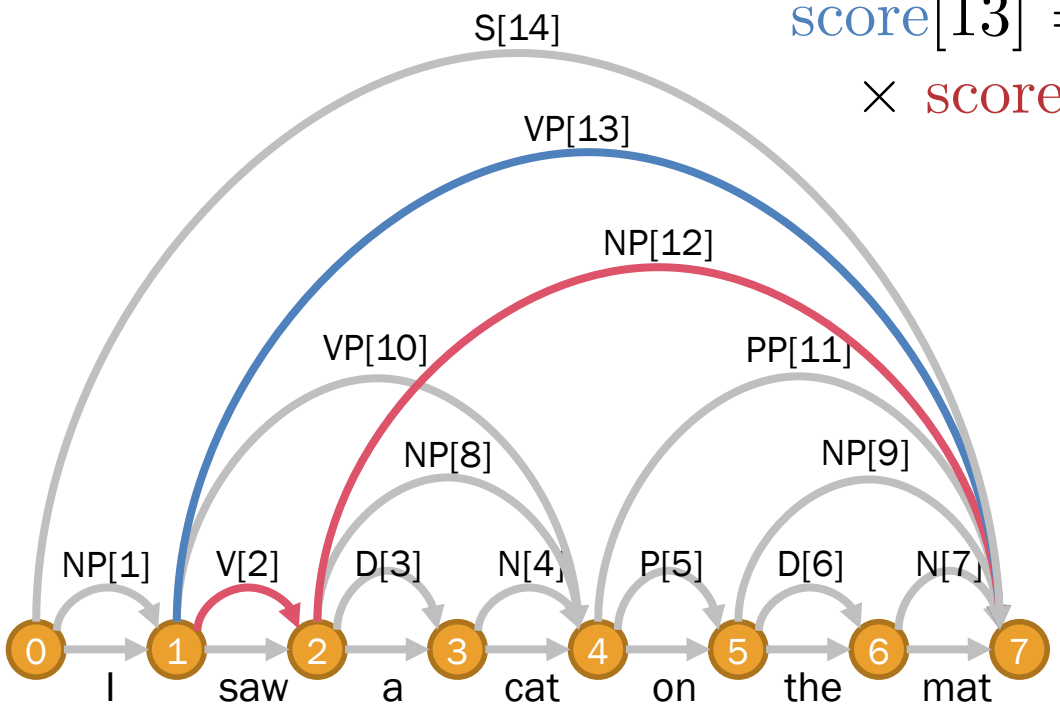
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] [10, 11]	
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[13] = P(\text{VP} \rightarrow \text{V NP}) \times \text{score}[2] \times \text{score}[12]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

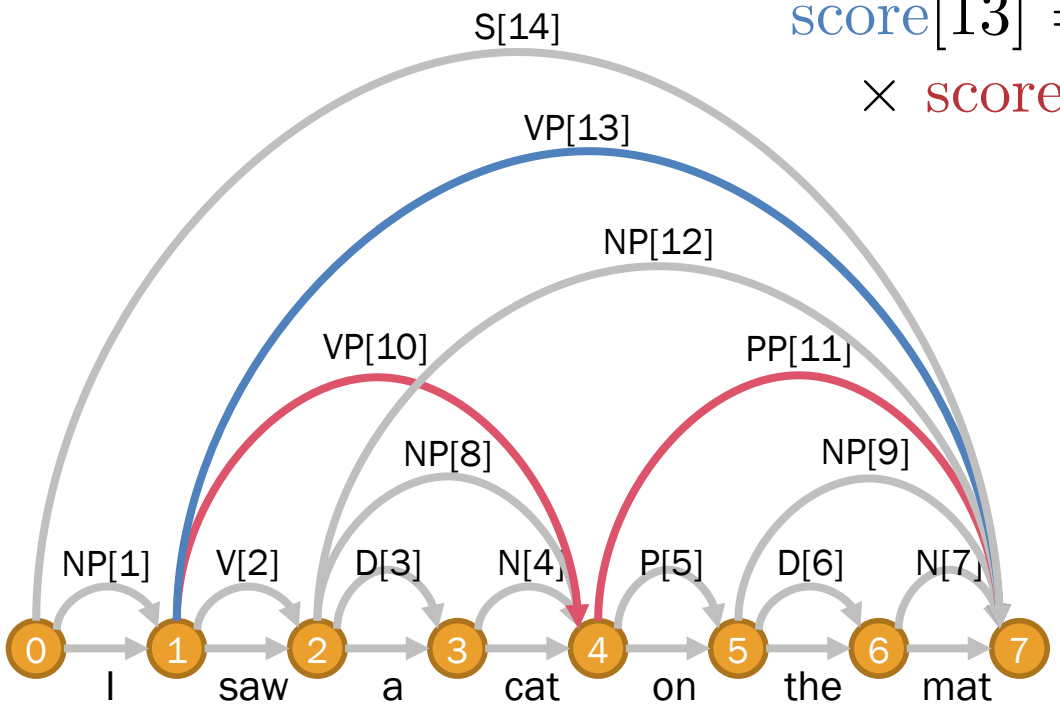
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] [10, 11]	0.0006
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[13] = P(\text{VP} \rightarrow \text{VP PP}) \times \text{score}[10] \times \text{score}[11]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

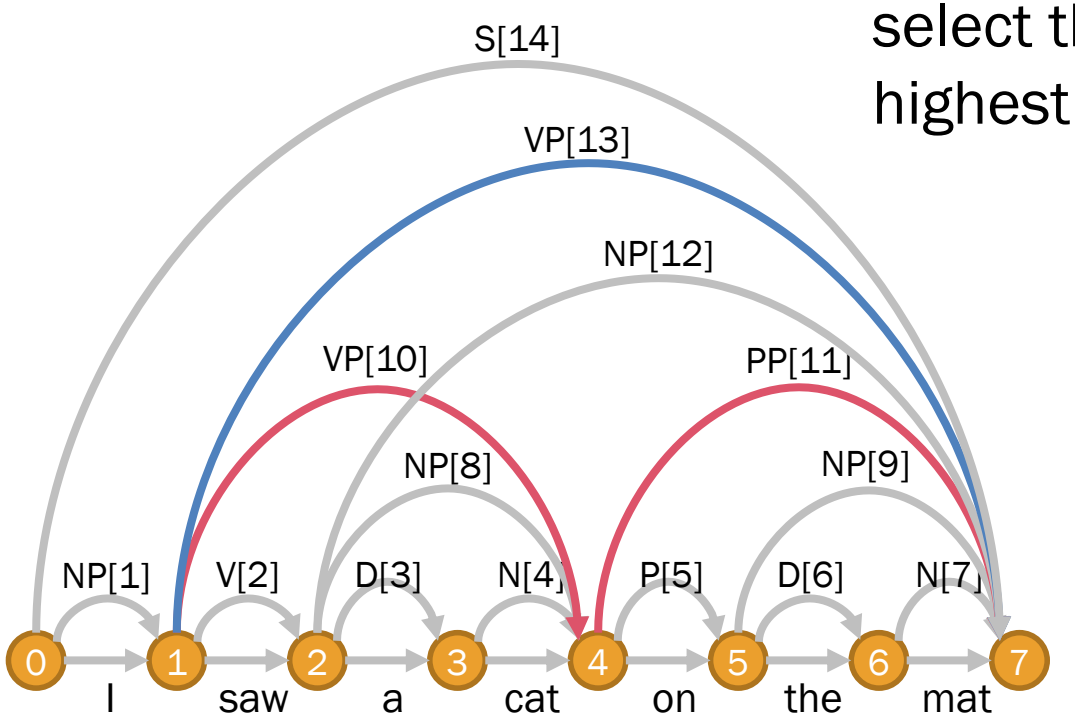
Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] [10, 11]	0.0006 0.0005
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

In the case of ambiguity, select the branch with highest score



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

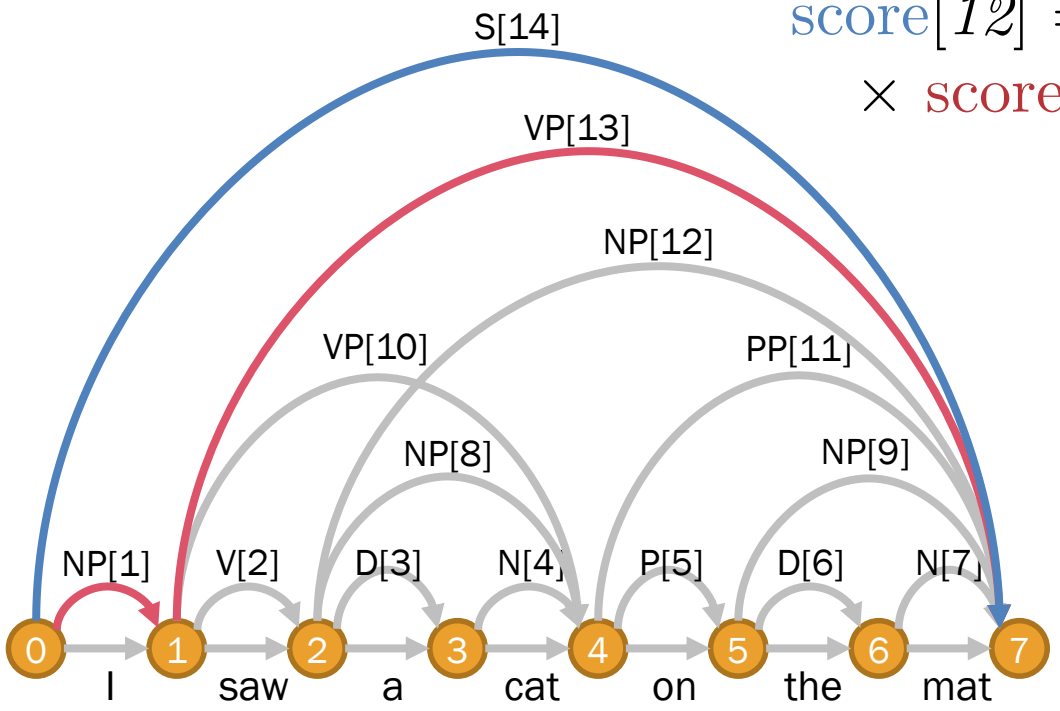
Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] ~~[10, 11]~~	0.0006 ~~0.0005~~
14	S → [1, 13]	

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”

Content: $A[m] \rightarrow B[d_1]\ C[d_2]$

$\text{score}[12] = P(\text{S} \rightarrow \text{NP VP}) \times \text{score}[1] \times \text{score}[13]$



Rules	Prob
NP → I	0.3
N → cat	0.4
N → mat	0.6
D → a	0.7
D → the	0.3
V → saw	1.0
P → on	1.0

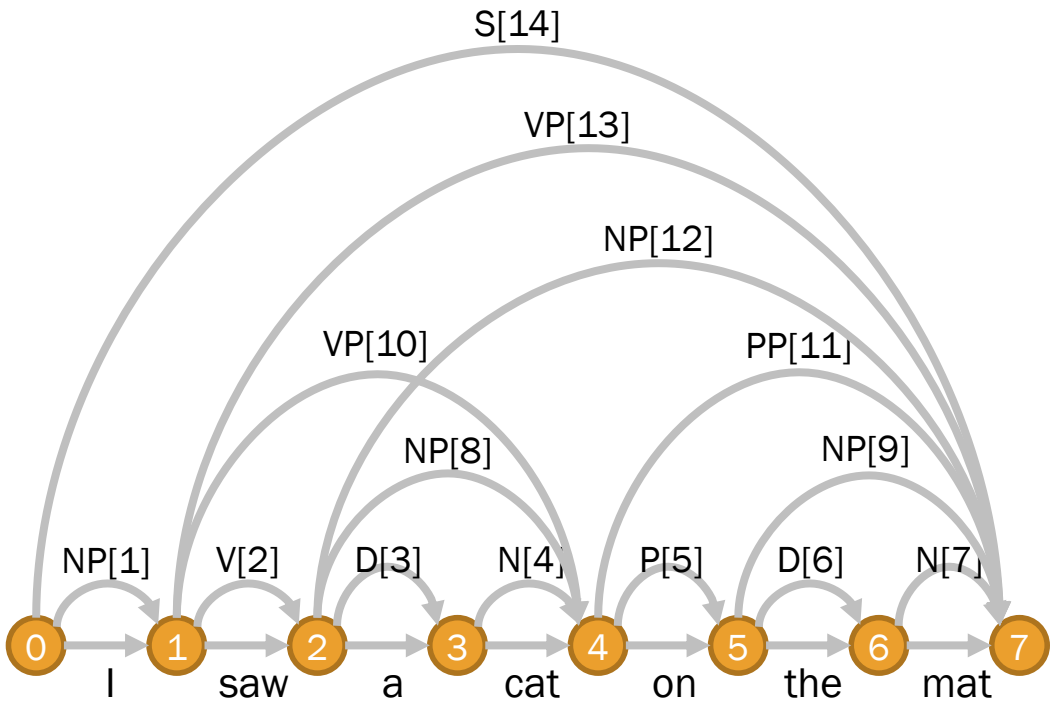
Rules	Prob
S → NP VP	1.0
NP → D N	0.2
NP → NP PP	0.5
VP → V NP	0.6
VP → VP PP	0.4
PP → P NP	1.0

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] ~~[10, 11]~~	0.0006 ~~0.0005~~
14	S → [1, 13]	0.0002

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



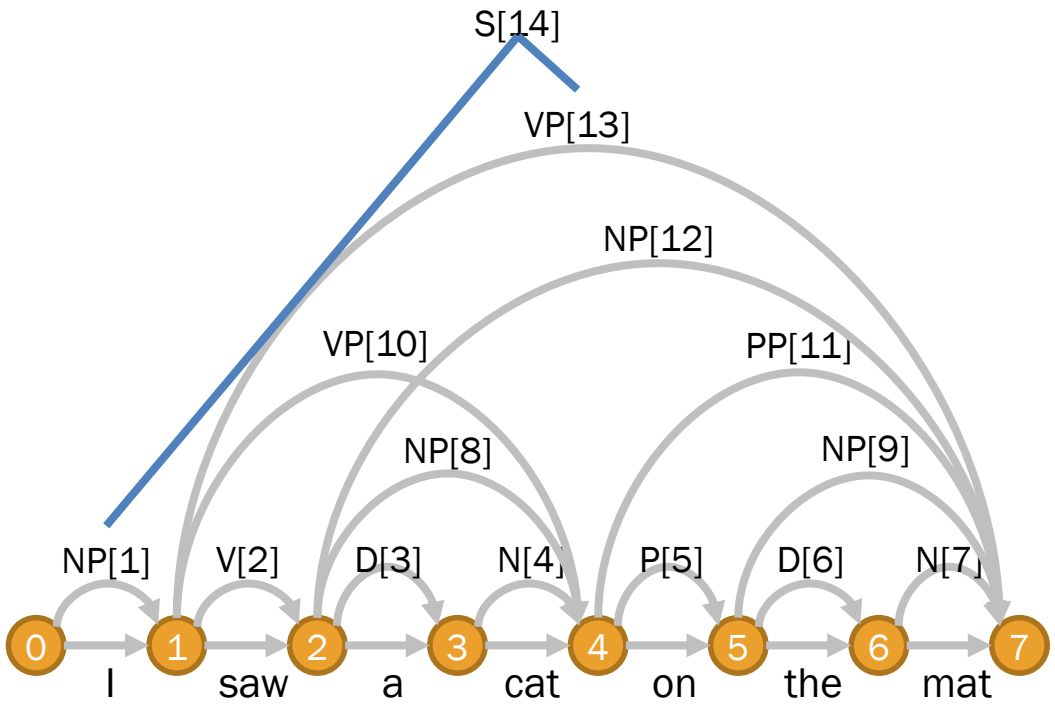
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] ~~[10, 11]~~	0.0006 ~~0.0005~~
14	S → [1, 13]	0.0002

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



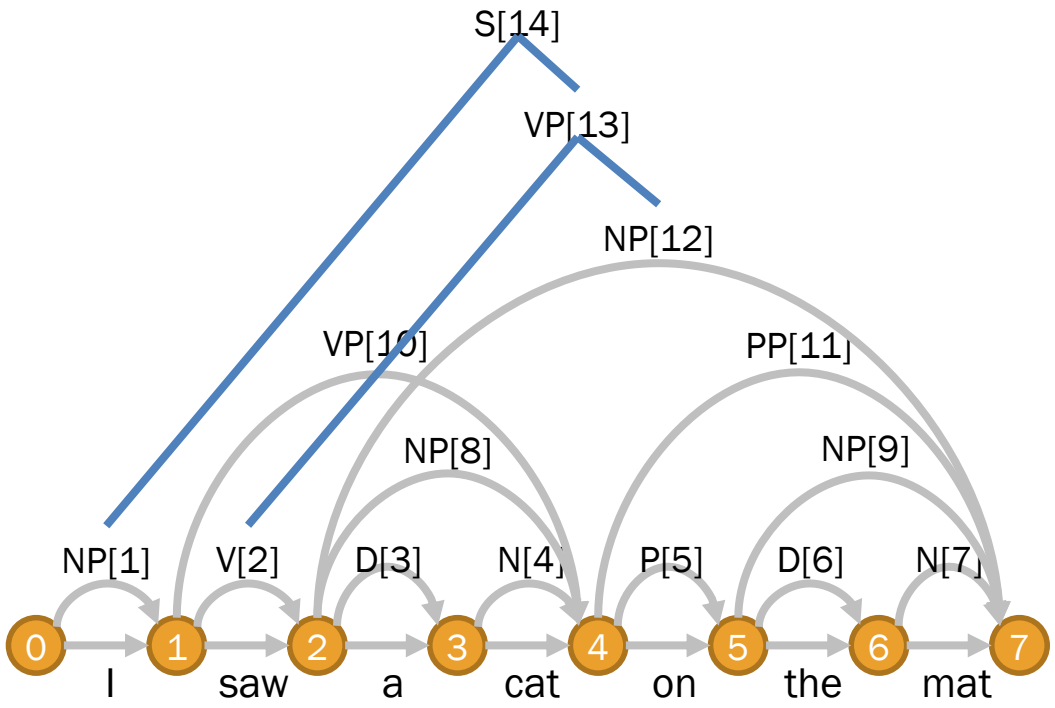
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	VP → [2, 12] ~~[10, 11]~~	0.0006 ~~0.0005~~
14	**S → [1, 13]**	**0.0002**

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



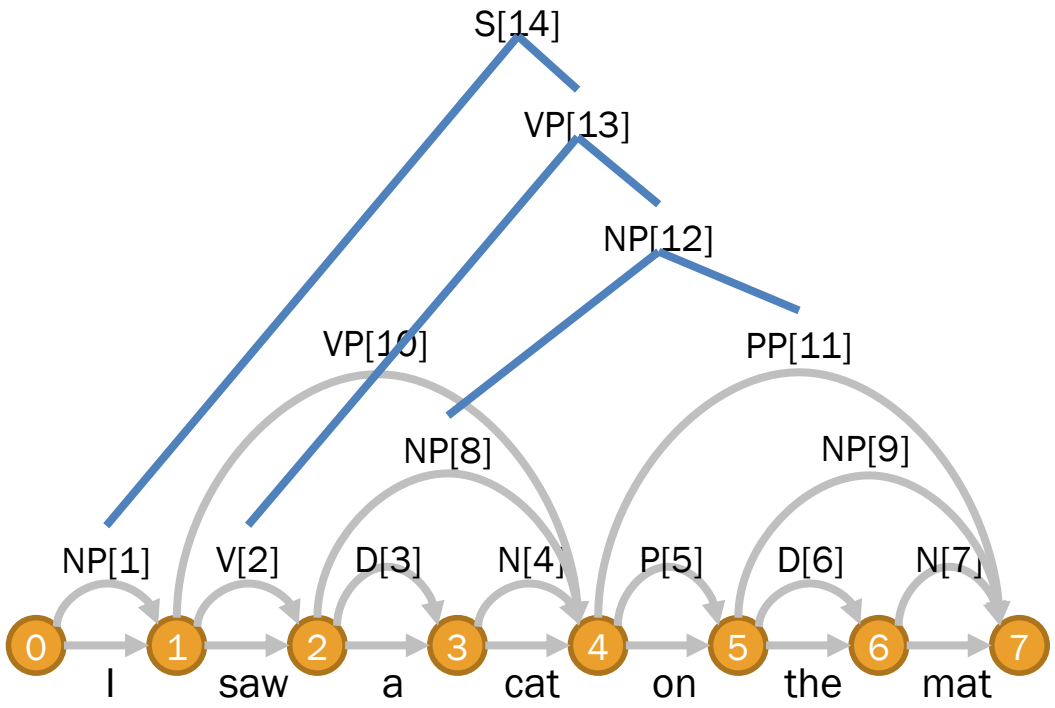
Decoding: Traverse top-down from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	NP → [8, 11]	0.0010
13	**VP → [2, 12]** *~~[10, 11]~~*	**0.0006** ~~0.0005~~
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



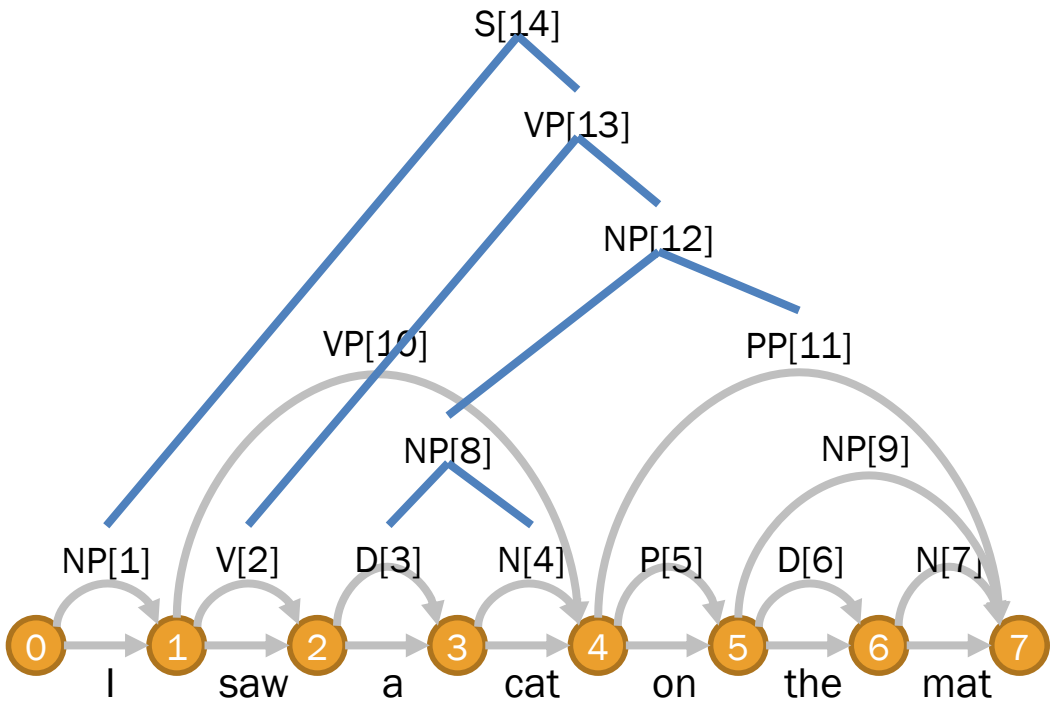
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
12	**NP → [8, 11]**	**0.0010**
~~13~~	~~VP → [2, 12]~~ *[10, 11]*	~~0.0006~~ ~~0.0005~~
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



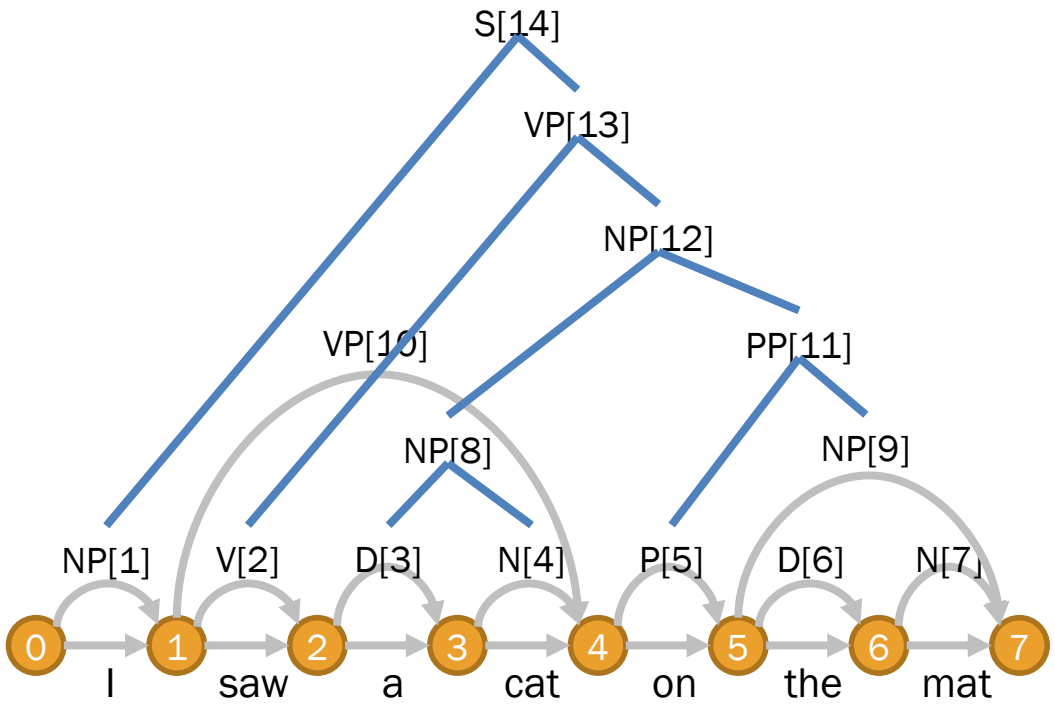
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
8	NP → [3, 4]	0.0556
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	PP → [5, 9]	0.0360
~~12~~	~~NP → [8, 11]~~	~~0.0010~~
13	~~VP → [2, 12]~~ *~~[10, 11]~~*	~~0.0006~~ ~~0.0005~~
14	S → [1, 13]	0.0002

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



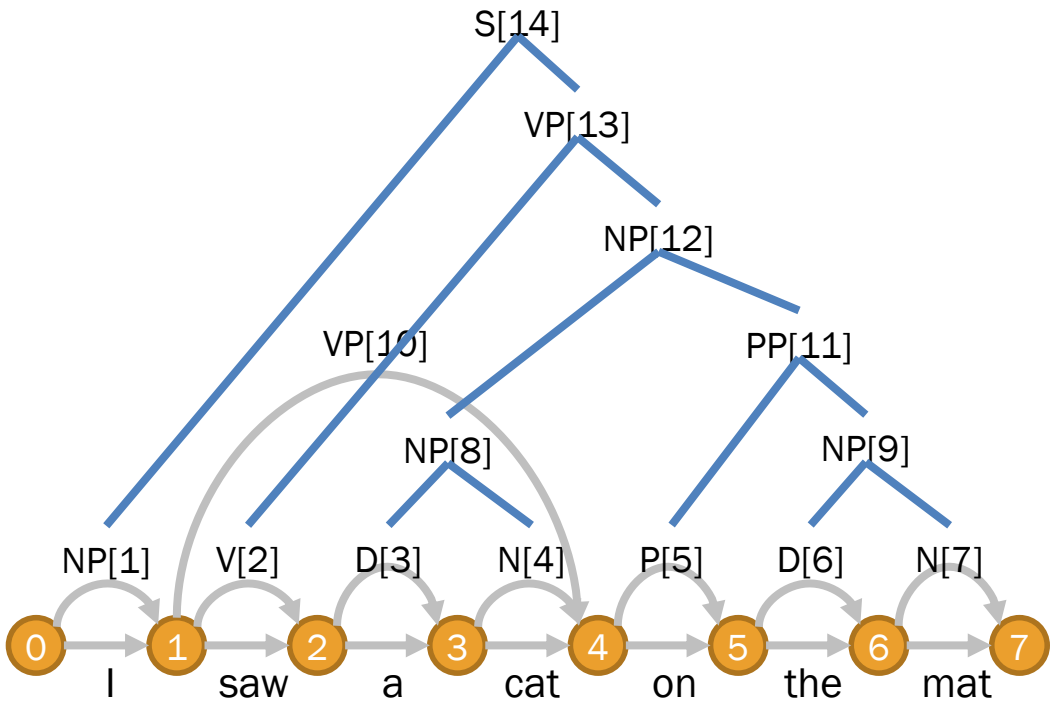
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
~~8~~	~~NP → [3, 4]~~	~~0.0556~~
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
11	**PP → [5, 9]**	**0.0360**
~~12~~	~~NP → [8, 11]~~	~~0.0010~~
~~13~~	~~VP → [2, 12]~~ *~~[10, 11]~~*	~~0.0006~~ ~~0.0005~~
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



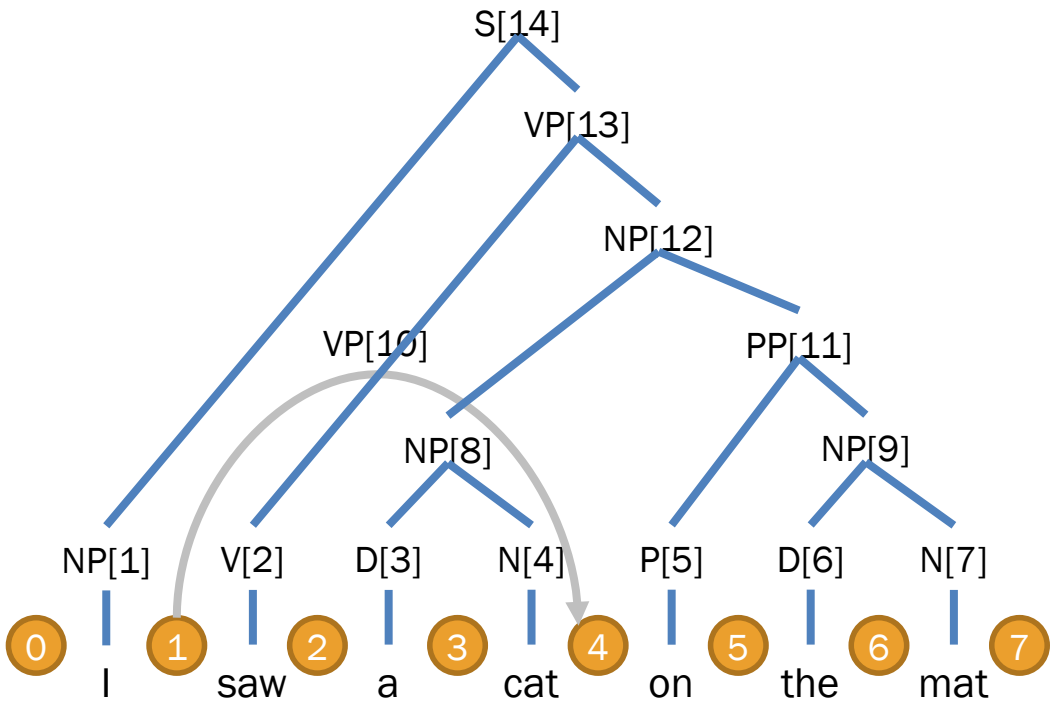
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
~~8~~	~~NP → [3, 4]~~	~~0.0556~~
9	NP → [6, 7]	0.0360
10	VP → [2, 8]	0.0336
~~11~~	~~PP → [5, 9]~~	~~0.0360~~
~~12~~	~~NP → [8, 11]~~	~~0.0010~~
~~13~~	~~VP → [2, 12]~~ *~~[10, 11]~~*	~~0.0006~~ ~~0.0005~~
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



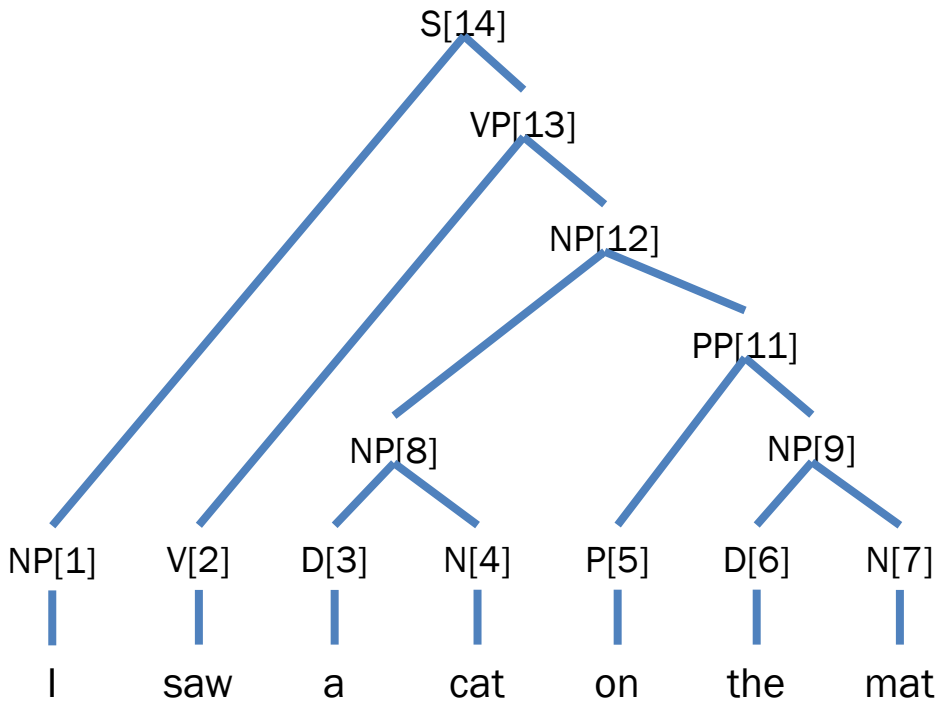
Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
1	NP → I	0.3
2	V → saw	1.0
3	D → a	0.7
4	N → cat	0.4
5	P → on	1.0
6	D → the	0.3
7	N → mat	0.6

Item	Content	Score
~~8~~	~~NP → [3, 4]~~	~~0.0556~~
~~9~~	~~NP → [6, 7]~~	~~0.0360~~
10	VP → [2, 8]	0.0336
~~11~~	~~PP → [5, 9]~~	~~0.0360~~
~~12~~	~~NP → [8, 11]~~	~~0.0010~~
~~13~~	~~VP → [2, 12]~~ *~~[10, 11]~~*	~~0.0006~~ ~~0.0005~~
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Viterbi Algorithm

- E.g. Parsing “I saw a cat on the mat”



Decoding: Traverse backward from the last item and create branching along the way

Item	Content	Score
~~1~~	~~NP → I~~	~~0.3~~
~~2~~	~~V → saw~~	~~1.0~~
~~3~~	~~D → a~~	~~0.7~~
~~4~~	~~N → cat~~	~~0.4~~
~~5~~	~~P → on~~	~~1.0~~
~~6~~	~~D → the~~	~~0.3~~
~~7~~	~~N → mat~~	~~0.6~~

Item	Content	Score
~~8~~	~~NP → [3, 4]~~	~~0.0556~~
~~9~~	~~NP → [6, 7]~~	~~0.0360~~
10	VP → [2, 8]	0.0336
~~11~~	~~PP → [5, 9]~~	~~0.0360~~
~~12~~	~~NP → [8, 11]~~	~~0.0010~~
~~13~~	~~VP → [2, 12]~~ *[10, 11]*	~~0.0006~~ 0.0005
~~14~~	~~S → [1, 13]~~	~~0.0002~~

Noise Reduction by Variational Bayesian

(Attias, 2000; Beal and Ghahramani, 2003)

Variational Bayesian EM (VB-EM)

- In traditional EM, M-Step is sensitive to noise
  - Arithmetic division does not discern expectation values from noise ones
  - This is because traditional EM deals with the likelihood $P(\mathcal{D}|\Theta)$ rather than the posterior probability $P(\Theta|\mathcal{D})$
- VB-EM directly deals with the posterior
  - If we can maximize $P(\mathcal{D})$, finding the posteriori probability will become tractable

Dealing with the Posterior Probability

- We factorize $P(\boldsymbol{\Theta}, \mathbf{Z}|\mathcal{D})$ with mean-field approximation

$$P(\boldsymbol{\Theta}, \mathbf{Z}|\mathcal{D}) \approx Q(\boldsymbol{\Theta}, \mathbf{Z})$$
$$= Q(\boldsymbol{\Theta})Q(\mathbf{Z})$$

  where $Q(\boldsymbol{\Theta})$ and $Q(\mathbf{Z})$ are **variational** distributions [*adj.* relating to small changes in the values]

- By Jensen's inequality, we obtain the lower bound of log $P(\mathcal{D})$

$$\log P(\mathcal{D}) \geq \sum_{\Theta} \left[\mathrm{KL}(Q(\Theta)||P(\Theta)) + \sum_{\mathbf{Z}} \mathrm{KL}(Q(\mathbf{Z})||P(D, \mathbf{Z}|\Theta)) \right]$$
$$= \mathcal{F}(Q(\Theta), Q(\mathbf{Z}))$$

($\mathcal{F}$ = free /residual energy)

VB-EM Algorithm

- **VB-E Step:** We compute the variational distribution of missing values $\mathbf{Z}$ by

$$Q(\mathbf{Z}) \quad \propto \quad \exp \mathbb{E}_{Q(\Theta)} \left[\log P(\mathcal{D}, \mathbf{Z} | \Theta) \right]$$

- **VB-M Step:** We compute the variational distribution of the parameters Θ by

$$Q(\Theta) \quad \propto \quad P(\Theta) \exp \mathbb{E}_{Q(\mathbf{Z})} \left[\log P(\mathcal{D}, \mathbf{Z} | \Theta) \right]$$

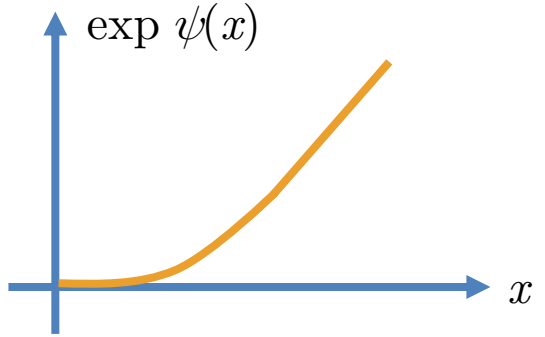
VB-EM Algorithm in Practice

- We simply replace every arithmetic division in EM with the following formula

$$P(y|x) = \frac{\theta_{xy} + \mu_{xy}}{\sum_x \theta_{xy} + \sum_x \mu_{xy}}$$

$$P(y|x) = \frac{\exp \psi[\theta_{xy} + \mu_{xy}]}{\exp \psi[\sum_x \theta_{xy} + \sum_x \mu_{xy}]}$$

- Each θ_{xy} is a model parameter
- Each μ_{xy} is a prior of θ_{xy}
- The **digamma function** ψ can be seen as the differentiation of the factorial function



Priors

- Larger priors make the models more tolerant to noise
  - Traditional EM is very sensitive to noise: the parameters for unseen observations will always be set to zero
  - VB-EM is tolerant to noise: the parameters for unseen observations will not be zero
- Too large priors may over-smooth the models as the value for each x approaches $\mu_x / \sum_{\mathrm{y}} \mu_y$

Conclusion

Conclusion

- Sequence prediction
  - Hidden Markov Models
  - Forward-Backward Algorithm
- Tree prediction
  - Probabilistic Context-Free Grammars
  - Inside-Outside Algorithm
- Noise reduction by Variational Bayes

Thank You

prachya@siit.tu.ac.th

kaamanita@gmail.com